

Objetos e Componentes Distribuídos

- 8.1 Introdução
- 8.2 Objetos distribuídos
- 8.3 Estudo de caso: CORBA
- 8.4 De objetos a componentes
- 8.5 Estudos de caso: Enterprise JavaBeans e Fractal
- 8.6 Resumo

Uma solução de *middleware* completa deve apresentar uma abstração de programação de nível mais alto e abstrair as complexidades subjacentes envolvidas nos sistemas distribuídos. Este capítulo examina duas das abstrações de programação mais importantes – objetos distribuídos e componentes – e examina as plataformas de *middleware* associadas, incluindo CORBA, Enterprise JavaBeans e Fractal.

O CORBA é um *middleware* que permite às aplicações se comunicarem umas com as outras independentemente de suas linguagens de programação, de suas plataformas de *hardware* e *software*, das redes pelas quais se comunicam e de seus desenvolvedores. As aplicações são construídas a partir de objetos CORBA, os quais implementam interfaces determinadas pela linguagem de definição de interface, IDL (Interface Description Language), do CORBA. Assim como RMI Java, o CORBA suporta invocação transparente de métodos em objetos remotos. O componente de *middleware* que suporta RMI é chamado de Object Request Broker ou ORB.

O *middleware* baseado em componentes surgiu como uma evolução natural dos objetos distribuídos, fornecendo suporte para gerenciamento de dependências entre componentes, ocultando os detalhes de baixo nível associados ao *middleware*, gerenciando as complexidades das aplicações distribuídas com propriedades não funcionais apropriadas (como a segurança) e suportando estratégias de distribuição adequadas. As principais tecnologias nessa área incluem o Enterprise JavaBeans e o Fractal.

8.1 Introdução

Os capítulos anteriores apresentaram os blocos de construção subjacentes fundamentais dos sistemas distribuídos, em termos de comunicação e de sistema operacional. Este capítulo se concentra nas soluções de *middleware* completas, apresentando os objetos distribuídos e componentes como dois dos estilos de *middleware* mais importantes em uso atualmente. Essa discussão é seguida, nos Capítulos 9 e 10, pela consideração de estratégias alternativas, baseadas em serviços Web e soluções *peer-to-peer*.

Conforme discutido no Capítulo 2, a tarefa do *middleware* é fornecer uma abstração de programação de nível mais alto para o desenvolvimento de sistemas distribuídos e, por meio de camadas, abstrair a heterogeneidade da infraestrutura subjacente a fim de promover a operação conjunta e a portabilidade.

Middleware de objetos distribuídos • A principal característica dos objetos distribuídos é que eles permitem adotar um modelo de programação orientada a objetos para o desenvolvimento de sistemas distribuídos e, por meio disso, ocultar a complexidade da programação distribuída. Nessa estratégia, as entidades que se comunicam são representadas por objetos. Os objetos se comunicam usando, principalmente, invocação de método remoto, mas possivelmente usando também um paradigma de comunicação alternativo (como os eventos distribuídos). Essa estratégia relativamente simples tem diversas vantagens importantes, incluindo:

- O *encapsulamento* inerente às soluções baseadas em objetos é apropriado à programação distribuída.
- A propriedade relacionada da *abstração de dados* oferece uma clara separação entre a especificação de um objeto e sua implementação, permitindo aos programadores lidar exclusivamente com as interfaces e não se preocupar com os detalhes da implementação, como a linguagem de programação e o sistema operacional usados.
- Essa estratégia também proporciona soluções mais *dinâmicas* e *extensíveis*, permitindo, por exemplo, a introdução de novos objetos ou a substituição de um objeto por outro (compatível).

Estão disponíveis diversas soluções de *middleware* baseadas em objetos distribuídos, incluindo RMI Java e CORBA. Resumimos as principais características dos objetos distribuídos na Seção 8.2 e fornecemos um estudo de caso detalhado do CORBA na Seção 8.3.

Middleware baseado em componentes • As soluções baseadas em componentes foram desenvolvidas para superar diversas limitações observadas pelos desenvolvedores de aplicativo que trabalham com *middleware* de objeto distribuído:

Dependências implícitas: as interfaces de objeto não descrevem as dependências da implementação de um objeto, tornando difícil desenvolver sistemas baseados em objetos (especialmente por parte de desenvolvedores externos) e, subsequentemente, gerenciá-los.

Complexidade de programação: a programação de *middleware* de objeto distribuído leva à necessidade de dominar muitos detalhes de baixo nível associados às implementações de *middleware*.

Falta de separação de aspectos relacionados à distribuição: os desenvolvedores de aplicativo são obrigados a considerar detalhes relacionados a segurança, falha de tratamento e concorrência, que são predominantemente semelhantes de um aplicativo para outro.

Ausência de suporte para implantação: o *middleware* baseado em objeto fornece pouco ou nenhum suporte para a implantação e definição de configurações (potencialmente complexas) de objetos.

As soluções baseadas em componentes podem ser mais bem entendidas como uma evolução natural dos objetos, complementando a forte herança desse trabalho anterior. A Seção 8.4 discutirá essa argumentação com mais detalhes e apresentará as principais características de uma estratégia baseada em componentes. Em seguida, a Seção 8.5 apresentará dois estudos de caso contrastantes de soluções baseadas em componentes, Enterprise JavaBeans e Fractal, com o primeiro oferecendo uma solução abrangente que abstrai muitos dos principais problemas do desenvolvimento de aplicações distribuídas e o segundo representando uma solução mais leve, frequentemente usada na construção de tecnologias de *middleware* mais complexas.

8.2 Objetos distribuídos

O *middleware* baseado em objetos distribuídos é projetado para fornecer um modelo de programação a partir de princípios orientados a objetos e, portanto, para trazer as vantagens da estratégia orientada a objetos para a programação distribuída.

Emmerich [2000] vê esses objetos distribuídos como uma evolução natural de três áreas de atividade:

- *Sistemas distribuídos:* os primeiros *middlewares* eram baseados no modelo cliente-servidor e havia o desejo por abstrações de programação mais sofisticadas.
- *Linguagens de programação:* o trabalho anterior em linguagens orientadas a objetos, como Simula-67 e Smalltalk, levou ao surgimento de outras, mais atuais e muito utilizadas, como Java e C++ (linguagens extensivamente usadas em sistemas distribuídos).
- *Engenharia de software:* foi feito um progresso significativo no desenvolvimento de métodos de projeto orientados a objetos, levando ao surgimento da UML (Unified Modelling Language) como notação padrão do setor para especificar sistemas de *software* orientados a objetos (potencialmente distribuídos).

Em outras palavras, por meio da adoção de uma estratégia orientada a objetos, os desenvolvedores de sistemas distribuídos não somente receberam abstrações de programação mais ricas (usando linguagens de programação conhecidas, como C++ e Java), como também puderam utilizar princípios de projeto, ferramentas e técnicas orientados a objetos (incluindo a UML) no desenvolvimento de *software* de sistemas distribuídos. Isso representa um grande avanço em uma área em que, anteriormente, não havia tais técnicas de projeto. É interessante notar que o OMG (Object Management Group), a organização que desenvolveu o CORBA (veja a Seção 8.3) também gerencia a padronização da UML.

O *middleware de objeto distribuído* oferece uma abstração de programação baseada em princípios orientados a objetos. Os principais exemplos de *middleware* de objeto distribuído incluem RMI Java (discutida na Seção 5.5) e CORBA (examinado em profundidade na Seção 8.3, a seguir). Embora RMI Java e CORBA tenham muito em comum, há uma diferença importante: o uso de RMI Java é restrito ao desenvolvimento baseado em Java, enquanto o CORBA é uma solução multilinguagem que permite a operação conjunta de objetos escritos em diversas linguagens. (Existe suporte para C++, Java, Python e várias outras linguagens.)

<i>Objetos</i>	<i>Objetos distribuídos</i>	<i>Descrição de objeto distribuído</i>
Referências de objeto	Referências de objeto remoto	Referências globalmente exclusivas para um objeto distribuído; podem ser passadas como parâmetro.
Interfaces	Interfaces remotas	Fornecem uma especificação abstrata dos métodos que podem ser invocados no objeto remoto; especificadas usando-se uma linguagem de definição de interface (IDL, Interface Definition Language).
Ações	Ações distribuídas	Iniciadas por uma invocação de método, potencialmente resultando em encadeamentos de invocação; as invocações remotas usam RMI.
Exceções	Exceções distribuídas	Exceções adicionais geradas a partir da natureza distribuída do sistema, incluindo perda de mensagens ou falha de processos.
Coleta de lixo	Coleta de lixo distribuída	Esquema ampliado para garantir que um objeto continue a existir, caso exista pelo menos uma referência de objeto ou referência de objeto remoto para esse objeto; caso contrário, ele deve ser removido. Exige um algoritmo de coleta de lixo distribuída.

Figura 8.1 Objetos distribuídos.

Deve-se enfatizar que a programação com objetos distribuídos é diferente e significativamente mais complexa que a programação orientada a objetos padrão, conforme resumido a seguir.

As diferenças: as principais diferenças entre objetos e objetos distribuídos já foram abordadas na Seção 5.4.1, no contexto de RMI. Por conveniência, essa discussão está repetida aqui de forma resumida (veja a Figura 8.1). Outras diferenças surgirão quando examinarmos o CORBA em detalhes, na Seção 8.3. Elas incluem:

- *Classe* é um conceito fundamental nas linguagens orientadas a objetos, mas não é uma característica muito destacada no *middleware* de objeto distribuído. Conforme observado no estudo de caso do CORBA, é difícil concordar com uma interpretação comum de classe em um ambiente heterogêneo, em que várias linguagens coexistem. De uma forma mais geral, no mundo orientado a objetos, a noção de classe tem várias interpretações, incluindo a descrição do comportamento associado a um grupo de objetos (o modelo usado para criar um objeto a partir da classe), o lugar para se instanciar um objeto com determinado comportamento (a fábrica associada) ou mesmo o grupo de objetos que obedece a esse comportamento. Embora o termo “classe” seja evitado, termos mais específicos, como “fábrica” e “modelo”, são usados sem dificuldade (uma fábrica é um objeto que instanciará um novo objeto a partir de determinado modelo).

- O estilo de *herança* é significantivamente diferente do oferecido na maioria das linguagens orientadas a objetos. Em particular, o *middleware* de objeto distribuído oferece *herança de interface*, que é uma relação entre interfaces por meio da qual a nova interface herda as assinaturas de método da interface original e pode adicionar outras. Em contraste, linguagens orientadas a objetos, como a Smalltalk, oferecem *herança de implementação* como uma relação entre implementações, por meio da qual a nova classe (neste caso) herda a implementação (e, portanto, o comportamento) da classe original e pode adicionar comportamento extra. É muito mais difícil implementatar herança de implementação, particularmente em sistemas distribuídos, devido à necessidade de solucionar o comportamento executável correto em tempo de execução. Considere, por exemplo, o nível de heterogeneidade que pode existir em um sistema distribuído, junto à necessidade de implementar soluções altamente escaláveis.

Wegner, em seu influente artigo sobre linguagens orientadas a objetos [Wegner 1987], define orientação a objetos como objeto + classe + herança. Nos sistemas distribuídos, a interpretação é claramente um pouco diferente, com classe e herança evitadas ou adaptadas. O que resta é um forte enfoque no encapsulamento e na abstração de dados, junto a poderosos vínculos com metodologias de projeto, conforme enfatizado anteriormente.

Complexidades adicionais: devido a complexidades adicionais envolvidas, o *middleware* de objeto distribuído associado deve fornecer mais funcionalidades, conforme resumido a seguir:

Comunicação entre objetos: uma estrutura de *middleware* de objeto distribuído deve fornecer um ou mais mecanismos para os objetos se comunicarem no ambiente distribuído. Normalmente, isso é fornecido pela invocação de método remoto, embora frequentemente o *middleware* de objeto distribuído complemente isso com outros paradigmas de comunicação (por exemplo, estratégias indiretas, como os eventos distribuídos). O CORBA fornece um serviço de eventos e um serviço de notificação associado, ambos implementados como serviços sobre o *middleware* básico (veja a Seção 8.3.4).

Gerenciamento de ciclo de vida: gerenciamento de ciclo de vida relaciona-se à criação, migração e exclusão de objetos, com cada etapa tendo de lidar com a natureza distribuída do ambiente subjacente.

Ativação e desativação: nas implementações não distribuídas, frequentemente pode-se supor que os objetos estão ativos o tempo todo, enquanto o processo que os contém é executado. Contudo, nos sistemas distribuídos, isso não pode ser presumido, pois os números de objetos podem ser muito grandes e, assim, seria desperdício de recursos ter todos os objetos disponíveis a todo momento. Além disso, nós que contêm objetos podem ficar indisponíveis por períodos de tempo. Ativação é o processo de tornar um objeto ativo no ambiente distribuído, fornecendo os recursos necessários para que ele processe as invocações recebidas – efetivamente, localizando o objeto na memória virtual e dando a ele as *threads* necessárias para executar. Desativação é o processo oposto, tornando um objeto temporariamente incapaz de processar invocações.

Persistência: normalmente, os objetos têm estado, e é importante manter esse estado entre os possíveis ciclos de ativação e desativação e em caso de falha do sistema. Portanto, o *middleware* de objeto distribuído deve oferecer gerenciamento de persistência para objetos com estado.

Serviços adicionais: uma estrutura de *middleware* de objeto distribuído abrangente também deve fornecer suporte para os diversos serviços de sistema distribuído considerados neste livro, incluindo serviços de atribuição de nomes, segurança e transação.

8.3 Estudo de caso: CORBA

O OMG (Object Management Group) foi formado em 1989, com o objetivo de estimular a adoção de sistemas de objetos distribuídos para usufruir as vantagens da programação orientada a objetos no desenvolvimento de *software* para sistemas distribuídos, os quais estavam se tornando muito difundidos. Para atingir seus objetivos, o OMG defendeu o uso de sistemas abertos baseados em interfaces orientadas a objetos padrão. Esses sistemas seriam construídos a partir de *hardware*, redes de computadores, sistemas operacionais e linguagens de programação heterogêneas.

Uma motivação importante foi permitir que os objetos distribuídos fossem implementados em qualquer linguagem de programação e pudessem se comunicar uns com os outros. Portanto, foi projetada uma linguagem de interface independente de qualquer linguagem de implementação específica.

Foi introduzida uma metáfora, o ORB (ou object request broker – agente de requisição de objeto), cuja função é ajudar um cliente a invocar um método em um objeto (seguindo o estilo RMI, conforme apresentado no Capítulo 5). Essa função envolve localizar o objeto, invocá-lo se necessário e, então, comunicar a requisição do cliente para o objeto, o qual o executa e responde.

Esta seção apresenta um estudo de caso do CORBA (Common Object Request Broker Architecture) do OMG, complementando esse conceito de agente de requisição de objeto. A apresentação se concentra na especificação CORBA 2 (a principal inovação em seu sucessor, CORBA 3, é a introdução de um modelo de componente, conforme discutido na Seção 8.4)

Os principais componentes do *framework* de RMI CORBA independente de linguagem são:

- Uma linguagem de definição de interface conhecida como IDL, que será descrita com detalhes na Seção 8.3.1.
- Uma arquitetura, que será discutida na Seção 8.3.2.
- Uma representação externa de dados, chamada CDR, que foi descrita na Seção 4.3.1. Ela também define formatos específicos para as mensagens em um protocolo de requisição-resposta. Além das mensagens de requisição-resposta, ela especifica mensagens para fazer perguntas sobre a localização de um objeto, cancelar requisições e relatar erros.
- Uma forma padrão para referências de objeto remoto, que será descrita na Seção 8.3.3.

A arquitetura CORBA também admite serviços CORBA – um conjunto de serviços genéricos úteis para aplicações distribuídas. Eles serão apresentados brevemente na Seção 8.3.4 (uma versão mais completa deste estudo de caso, incluindo uma consideração detalhada dos serviços do CORBA, pode ser encontrada no *site* que acompanha o livro [www.cdk5.net] – em inglês).

A Seção 8.3.5 também contém um exemplo de desenvolvimento de cliente-servidor usando CORBA.

Para um conjunto interessante de artigos sobre CORBA, consulte a edição especial do *CACM* [Seetharamanan 1998].

8.3.1 RMI CORBA

Programar em um sistema RMI de multilinguagens, como RMI CORBA, exige mais do programador se comparado com o desenvolvimento em um sistema RMI de linguagem única, como RMI Java. Os seguintes novos conceitos precisam ser aprendidos:

- o modelo de objeto oferecido pelo CORBA;
- a linguagem de definição de interface;
- seu mapeamento para a linguagem de implementação.

Outros aspectos da programação em CORBA são semelhantes aos discutidos no Capítulo 5. Em particular, o programador define interfaces remotas para os objetos remotos e depois usa um compilador de interface para produzir os *proxies* e esqueletos correspondentes. No entanto, no CORBA, os *proxies* são gerados na linguagem do cliente e os esqueletos, na linguagem do servidor.

Modelo de objeto do CORBA • O modelo de objeto do CORBA é semelhante àquele descrito na Seção 5.4.1, mas os clientes não são necessariamente objetos – um cliente pode ser qualquer programa que envie mensagens de requisição para objetos remotos e receba respostas. O termo *objeto CORBA* é usado para se referir aos objetos remotos. Assim, um objeto CORBA implementa uma interface IDL, tem uma referência de objeto remoto e é capaz de responder às invocações de métodos em sua interface IDL. Um objeto CORBA pode, por exemplo, ser implementado por uma linguagem não orientada a objeto, sem o conceito de classe. Como as linguagens de implementação terão diferentes noções de classe, ou mesmo nenhuma noção, o conceito de classe não existe no CORBA (veja também a discussão na Seção 8.2). Portanto, classes não podem ser definidas na IDL do CORBA, o que significa que instâncias de classes não podem ser passadas como argumentos. Entretanto, estruturas de dados de vários tipos e complexidade arbitrária podem ser passadas como argumentos.

IDL CORBA • A IDL do CORBA especifica um nome e um conjunto de métodos que os clientes podem solicitar. Como um exemplo inicial, a Figura 8.2 mostra duas interfaces, chamadas *Shape* (linha 3) e *ShapeList* (linha 5), que são versões de IDL das interfaces definidas na Figura 5.16. Elas são precedidas por definições de duas estruturas (*structs*), as quais são usadas como tipos de parâmetro na definição dos métodos. Note, em particular, que *GraphicalObject* é definido como *struct*, embora fosse uma classe no exemplo de RMI em Java. Um componente cujo tipo é *struct* tem um conjunto de campos contendo valores de diferentes tipos, como as variáveis de instância de um objeto, mas não tem métodos.

Mais detalhadamente, a linguagem de definição de interface – ou simplesmente IDL – CORBA fornece recursos para a definição de módulos, interfaces, tipos, atributos e assinaturas de método. Podemos ver exemplos de tudo isso, fora os módulos, nas Figuras 5.8 e 8.2. A IDL CORBA tem as mesmas regras léxicas da linguagem C++, mas possui palavras-chave adicionais para suportar distribuição, por exemplo, *interface*, *any*, *attribute*, *in*, *out*, *inout*, *readonly* e *raises*. Ela também permite recursos de pré-processamento padrão da linguagem C++. Veja, por exemplo, o *typedef* para *All* na Figura 8.3.

A gramática da IDL é um subconjunto da linguagem C++ ANSI, com construções adicionais para suportar assinaturas de método. Fornecemos, aqui, apenas um


```

struct Rectangle{                                1
    long width;
    long height;
    long x;
    long y;
};
struct GraphicalObject {                        2
    string type;
    Rectangle enclosing;
    boolean isFilled;
};
interface Shape {                               3
    long getVersion();
    GraphicalObject getAllState(); // retorna o estado do objeto GraphicalObject
};
typedef sequence <Shape, 100> All;              4
interface ShapeList {                          5
    exception FullException{ };                6
    Shape newShape(in GraphicalObject g) raises (FullException); 7
    All allShapes(); // retorna a sequência de referências de objeto remoto 8
    long getVersion();
};

```

Figura 8.2 Interfaces IDL *Shape* e *ShapeList*.

breve panorama da IDL. Uma visão geral útil e exemplos são dados em Baker [1997] e Henning e Vinoski [1999]. A especificação completa está disponível no *site* do OMG [OMG 2002a].

Módulos IDL: a construção do módulo permite que as interfaces e outras definições de tipo da IDL sejam agrupadas em unidades lógicas. Um *módulo* define um escopo de atribuição de nomes, o qual impede que os nomes definidos dentro dele colidam com os nomes definidos fora dele. Por exemplo, as definições das interfaces *Shape* e *ShapeList* poderiam pertencer a um módulo chamado *Whiteboard*, como mostrado na Figura 8.3.

Interfaces IDL: conforme já vimos, uma interface IDL descreve os métodos que estão disponíveis nos objetos CORBA que implementam essa interface. Os clientes de um objeto CORBA podem ser desenvolvidos apenas a partir do conhecimento de sua interface IDL. A partir do estudo de nossos exemplos, os leitores verão que uma interface define um conjunto de operações e atributos e, geralmente, depende de um conjunto de tipos definidos com ela. Por exemplo, a interface *PersonList*, na Figura 5.8, define um atributo e três métodos e depende do tipo *Person*.

Métodos IDL: a forma geral de uma assinatura de método é a seguinte:

```

[oneway] <tipo_retorno> <nome_método> (parâmetro1,...,parâmetroL)
    [raises (exceção1,..., exceçãoN)] [contexto (nome1,..., nomeM)]

```

onde as expressões entre colchetes são opcionais. Para um exemplo de assinatura de método que contenha apenas as partes exigidas, considere:

```

void getPerson(in string name, out Person p);

```

Os parâmetros são rotulados como *in*, *out* ou *inout*, onde o valor de um parâmetro *in* é passado do cliente para o objeto CORBA invocado e o valor de um parâmetro *out* é pas-


```

module Whiteboard {
    struct Rectangle{
        ...;
    };
    struct GraphicalObject {
        ...;
    };
    interface Shape {
        ...;
    };
    typedef sequence <Shape, 100> All;
    interface ShapeList {
        ...;
    };
};

```

Figura 8.3 Módulo de IDL *Whiteboard*.

sado de volta do objeto CORBA invocado para o cliente. Raramente são usados parâmetros rotulados como *inout*, mas eles indicam que o valor do parâmetro pode ser passado nas duas direções. O tipo de retorno pode ser especificado como *void*, caso nenhum valor deva ser retornado. A Figura 5.8 ilustra um exemplo simples do uso dessas palavras-chaves. Na Figura 8.2, linha 7, o parâmetro de *newShape* é um parâmetro *in*, para indicar que o argumento deve ser passado do cliente para o servidor na mensagem de requisição. O valor de retorno de um método fornece um parâmetro *out* adicional – ele pode ser indicado como *void*, caso não exista nenhum parâmetro *out*.

Os parâmetros podem ser qualquer um dos tipos primitivos, como *long* e *boolean*, ou um dos tipos construídos, como *struct* ou *array* (mais informações sobre tipos IDL primitivos e estruturados podem ser encontradas a seguir). Nosso exemplo mostra as definições de dois tipos *struct*, nas linhas 1 e 2. Sequências e vetores são definidos em *typedefs*, como se vê na linha 4, que mostra uma sequência de elementos de tipo *Shape* de tamanho 100. A semântica da passagem de parâmetros é a seguinte:

Passagem de objetos CORBA: qualquer parâmetro cujo tipo seja especificado pelo nome de uma interface IDL, como o valor de retorno *Shape* na linha 7, é uma referência para um objeto CORBA, e é passado o valor de uma referência de objeto remoto.

Passagem de tipos primitivos e construídos CORBA: os argumentos de tipos primitivos e construídos são copiados e passados por valor. Na chegada, um novo valor é criado no processo do destinatário. Por exemplo, o *struct GraphicalObject* passado como argumento (na linha 7) produz uma nova cópia desse *struct* no servidor.

Essas duas formas de passagem de parâmetro são combinadas no método *allShapes* (na linha 8), cujo tipo de retorno é um vetor de tipo *Shape* – isto é, um vetor de referências de objeto remoto. O valor de retorno é uma cópia do vetor na qual cada um dos elementos é uma referência de objeto remoto.

Semântica de invocação. A invocação remota no CORBA tem como padrão a semântica de chamada *no máximo uma vez*. Entretanto, a IDL pode especificar que a invocação de um método em particular tenha semântica *talvez*, usando a palavra-chave *oneway*. O cliente não é bloqueado em requisições *oneway*, que só podem ser usadas por métodos sem resultados. Para um exemplo de uma requisição *oneway*, veja o que lida com *callbacks* no final da Seção 8.3.5.

Exceções na IDL CORBA: a IDL CORBA permite que exceções sejam definidas nas interfaces e disparadas por seus métodos. A expressão opcional *raises* indica as exceções definidas pelo usuário que podem ser disparadas para terminar uma execução do método. Considere o exemplo a seguir da Figura 8.2:

```
exception FullException{ };
Shape newShape(in GraphicalObject g) raises (FullException);
```

Com a expressão *raises*, o método *newShape* especifica que ele pode disparar uma exceção chamada *FullException*, que está definida dentro da interface *ShapeList*. Em nosso exemplo, a exceção não contém variáveis. Entretanto, as exceções podem ser definidas para conter variáveis, por exemplo:

```
exception FullException {GraphicalObject g};
```

Quando uma exceção que contém variáveis é disparada, o servidor pode usar as variáveis para retornar informações para o cliente sobre o contexto da exceção.

O CORBA também pode produzir exceções de sistema relacionadas a problemas nos servidores (como o fato de estarem ocupados demais ou não poderem ativar objetos), problemas com a comunicação e problemas no lado do cliente. Os programas clientes devem tratar de exceções definidas pelo usuário e de sistema. A expressão opcional *context* é usada para fornecer mapeamentos de nomes de *string* para valores de *string*. Consulte Baker [1997] para uma explicação de *context*.

Tipos da IDL: a IDL suporta 15 tipos primitivos, os quais incluem *short* (16 bits), *long* (32 bits), *unsigned short*, *unsigned long*, *float* (32 bits), *double* (64 bits), *char*, *boolean* (TRUE, FALSE), *octet* (8 bits) e *any* (que pode representar qualquer tipo primitivo ou construído). Constantes da maioria dos tipos primitivos e *strings* podem ser declaradas usando-se a palavra-chave *const*. A IDL fornece um tipo especial chamado *Object*, cujos valores são referências de objeto remoto. Se um parâmetro ou resultado é de tipo *Object*, então o argumento correspondente pode se referir a qualquer objeto CORBA.

Os tipos construídos da IDL estão descritos na Figura 8.4, todos os quais são passados por valor em argumentos e resultados. Todos os vetores ou sequências usados como argumentos devem ser definidos em *typedefs*. Nenhum dos tipos de dados primitivos ou construídos pode conter referências.

O CORBA também suporta passagem por valor de objetos que não são CORBA [OMG 2002c]. Esses objetos que não são CORBA são parecidos com objetos, pois possuem atributos e métodos. Entretanto, eles são puramente objetos locais, pois suas operações não podem ser invocadas de forma remota. O recurso de passagem por valor fornece a capacidade de passar uma cópia de um objeto que não é CORBA entre cliente e servidor.

Isso é obtido pela adição, na IDL, de um tipo chamado *valuetype* para representar objetos que não são CORBA. Um *valuetype* é uma *struct* com assinaturas de método adicionais (como as de uma interface) e os argumentos e resultados *valuetype* são passados por valor. Isto é, o estado é passado para o *site* remoto e usado para produzir um novo objeto no destino.

Os métodos desse novo objeto podem ser invocados de forma local, fazendo seu estado divergir do estado do objeto original. Passar a implementação dos métodos não é tão simples assim, pois o cliente e o servidor podem usar linguagens diferentes. Entretanto, se o cliente e o servidor forem implementados em Java, o código poderá ser baixado

<i>Tipo</i>	<i>Exemplos</i>	<i>Uso</i>
<i>sequence</i>	<code>typedef sequence <Shape, 100> All;</code> <code>typedef sequence <Shape> All;</code> seqüências limitadas e não limitadas de <i>Shapes</i>	Define um tipo para uma seqüência de elementos de tamanho variável de um tipo IDL especificado. Pode ser especificado um limite superior para o tamanho.
<i>string</i>	<code>string name;</code> <code>typedef string<8></code> <code>SmallString;</code> seqüências limitadas e não limitadas de caracteres	Define uma seqüência de caracteres, terminada pelo caractere nulo. Pode ser especificado um limite superior para o comprimento.
<i>array</i>	<code>typedef octet uniqueId[12];</code> <code>typedef GraphicalObject G0[10][8];</code>	Define um tipo para uma seqüência multidimensional de elementos de tamanho fixo de um tipo IDL especificado.
<i>record</i>	<code>struct GraphicalObject {</code> <code>string type;</code> <code>Rectangle enclosing;</code> <code>boolean isFilled;</code> <code>};</code>	Define um tipo para um registro contendo um grupo de entidades relacionadas. <i>Structs</i> são passadas por valor em argumentos e resultados.
<i>enumerated</i>	<code>enum Rand</code> <code>(Exp, Number, Name);</code>	O tipo enumerado na IDL faz o mapeamento de um nome de tipo para um conjunto de valores inteiros.
<i>union</i>	<code>union Exp switch (Rand) {</code> <code>case Exp: string vote;</code> <code>case Number: long n;</code> <code>case Name: string s;</code> <code>};</code>	A união da IDL permite que um tipo de determinado conjunto de tipos seja passado como argumento. O cabeçalho é parametrizado por um <i>enum</i> , que especifica qual membro é usado.

Figura 8.4 Tipos construídos da IDL.

por *download*. Para uma implementação em C++ comum, o código necessário precisaria estar presente no cliente e no servidor.

Esse recurso é útil quando for vantajoso colocar uma cópia de um objeto no processo cliente para permitir que ele receba invocações locais. Entretanto, ele nem chega perto da passagem por valor de objetos CORBA.

Atributos: as interfaces IDL podem ter atributos, assim como métodos. Os atributos são como os campos de classe pública em Java. Eles podem ser definidos como *readonly*, onde for apropriado. Os atributos são privativos dos objetos CORBA, mas, para cada atributo declarado, dois métodos de acesso são gerados automaticamente pelo compilador de IDL: um para recuperar o valor do atributo e o outro para configurá-lo. Para atributos *readonly* é fornecido apenas o método de obtenção (*getter*). Por exemplo, a interface *PersonList* definida na Figura 5.2 inclui a seguinte definição de atributo:

readonly attribute string listname;

Herança: as interfaces IDL podem ser estendidas por meio de herança, conforme definido na Seção 8.2. Por exemplo, se a interface *B* estende a interface *A*, isso significa que ela pode adicionar novos tipos, constantes, exceções, métodos e atributos naqueles de *A*. Uma interface estendida pode redefinir tipos, constantes e exceções, mas não pode redefinir métodos. O valor de um tipo estendido é válido como valor de um parâmetro ou resultado do tipo ascendente. Por exemplo, o tipo *B* é válido como valor de um parâmetro

ou resultado do tipo *A*. Além disso, uma interface IDL pode estender mais de uma interface. Por exemplo, aqui, a interface *Z* estende *B* e *C*:

```
interface A { };
interface B: A { };
interface C { };
interface Z: B, C { };
```

Isso significa que *Z* tem todos os componentes de *B* e de *C* (além daqueles que ela redefine), assim como os que define como extensões.

Quando uma interface como *Z* estende mais de uma interface, existe a possibilidade de que ela possa herdar um tipo, constante ou exceção com o mesmo nome, de duas interfaces diferentes. Por exemplo, suponha que tanto *B* como *C* definam um tipo chamado *Q*; então, o uso de *Q* na interface *Z* será ambíguo, a menos que seja dado um nome com escopo, como *B::Q* ou *C::Q*. A IDL não permite que uma interface herde métodos ou atributos com nomes comuns de duas interfaces diferentes.

Todas as interfaces IDL herdam do tipo *Object*, o que implica que todas as interfaces IDL são compatíveis com o tipo *Object* – o que inclui referências de objeto remoto. Isso torna possível definir operações de IDL que podem receber como argumento ou retornar como resultado uma referência de objeto remoto de qualquer tipo. As operações *bind* e *resolve* no serviço de nomes (Naming Service) são exemplos disso.

Identificadores de tipo IDL: conforme veremos na Seção 8.3.2, os identificadores de tipo são gerados pelo compilador de IDL para cada tipo em uma interface IDL. Por exemplo, o tipo IDL para a interface *Shape* (Figura 8.3) poderia ser:

```
IDL:Whiteboard/Shape:1.0
```

Esse exemplo mostra que um nome de tipo da IDL tem três partes – o prefixo IDL, um nome de tipo e um número de versão. Como identificadores de interface são usados como chaves para acessar definições de interface no repositório de interfaces (descrito na Seção 8.3.2), os programadores devem garantir o fornecimento de um mapeamento único para as próprias interfaces. Os programadores podem usar o prefixo IDL *pragma* para prefixar um *string* adicional no nome de tipo, para distinguir seus próprios tipos dos de outros.

Diretivas pragma da IDL: elas permitem que propriedades adicionais que não pertencem à IDL sejam especificadas para componentes em uma interface IDL (consulte Henning e Vinoski [1999]). Essas propriedades incluem, por exemplo, especificar que uma interface será usada apenas de forma local ou fornecer o valor de um ID de repositório de interfaces. Cada *pragma* é introduzido por *#pragma* e especifica seu tipo, por exemplo:

```
#pragma version Whiteboard 2.3
```

Mapeamentos de linguagem CORBA • O mapeamento dos tipos na IDL para tipos de determinada linguagem de programação é muito simples. Por exemplo, em Java, os tipos primitivos na IDL são mapeados nos tipos primitivos correspondentes dessa linguagem. *Structs*, *enums* e *unions* são mapeados em classes Java; sequências e *arrays* na IDL são mapeados em vetores em Java. Uma exceção da IDL é mapeada em uma classe Java que fornece variáveis de instância para os campos da exceção e para os construtores. Os mapeamentos em C++ são semelhantemente simples.

Entretanto, surgem algumas dificuldades no mapeamento da semântica da passagem de parâmetros da IDL para a da linguagem Java. Em particular, a IDL permite que os

métodos retornem diversos valores separados por meio de parâmetros de saída, enquanto a linguagem Java só pode ter um único resultado. As classes *Holder* são fornecidas para resolver essa diferença, mas isso exige que o programador as utilize, o que não é nada simples. Por exemplo, o método *getPerson* da Figura 5.8 é definido na IDL como segue:

```
void getPerson(in string name, out Person p);
```

Em Java, o método equivalente seria definido como:

```
void getPerson(String name, PersonHolder p);
```

e o cliente teria de fornecer uma instância de *PersonHolder* como argumento de sua invocação. A classe portadora tem uma variável de instância que contém o valor do argumento para o cliente acessar por RMI quando a invocação retornar. Ela também tem métodos para transmitir o argumento entre servidor e cliente.

Embora as implementações em C++ do CORBA possam manipular parâmetros *out* e *inout* de forma bastante natural, os programadores de C++ sofrem com um conjunto de problemas diferentes nos parâmetros, relacionados ao gerenciamento do armazenamento. Essas dificuldades surgem quando referências de objeto e entidades de comprimento variável, como *strings* ou sequências, são passadas como argumentos.

Por exemplo, no Orbix [Baker 1997], o ORB mantém contagens de referência para objetos remotos e *proxies* e as libera quando elas não são mais necessárias. Ele fornece aos programadores métodos para liberá-las ou duplicá-las. Quando um método de servidor termina de executar, os argumentos e resultados *out* são liberados, e o programador deve duplicá-los, caso ainda sejam necessários. Por exemplo, um servente em C++ implementando a interface *ShapeList* precisará duplicar as referências retornadas pelo método *allShapes*. As referências de objeto passadas para os clientes devem ser liberadas quando não forem mais necessárias. Regras semelhantes se aplicam aos parâmetros de tamanho variável.

Em geral, os programadores que usam a IDL não apenas precisam aprender a própria notação da IDL como devem entender como seus parâmetros são mapeados nos parâmetros da linguagem de implementação.

RMI assíncrona • O CORBA suporta uma forma de RMI assíncrona que permite aos clientes fazerem invocações não bloqueantes nos objetos CORBA [OMG 2004e]. Ela se destina a ser implementada no cliente. Portanto, um servidor geralmente não sabe se ela é invocada de forma síncrona ou de forma assíncrona. (Uma exceção é o *Transaction Service*, que precisaria saber da diferença.)

A RMI assíncrona adiciona duas novas variantes na semântica de invocação das RMIs:

- *callback*, na qual um cliente usa um parâmetro extra para passar uma referência para uma *callback* com cada invocação, de modo que o servidor possa chamar de volta com os resultados;
- *consulta sequencial*, na qual o servidor retorna um objeto *valuetype*, que pode ser usado para fazer uma consulta sequencial ou esperar pela resposta.

A arquitetura da RMI assíncrona permite que um agente intermediário seja implantado para garantir que a requisição seja executada e, se necessário, armazene a resposta. Assim, ela é apropriada para uso em ambientes onde os clientes podem ser temporariamente desconectados – como, por exemplo, um cliente usando um *notebook* em um trem.

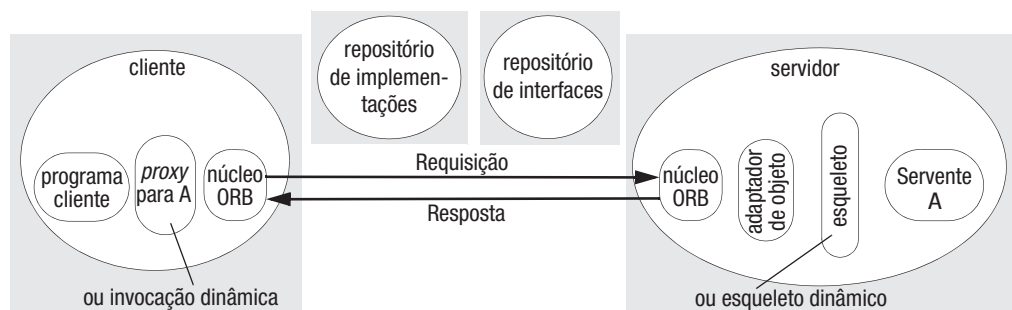


Figura 8.5 Os principais componentes da arquitetura CORBA.

8.3.2 A arquitetura CORBA

A arquitetura é projetada para suportar a função de um agente de requisição de objeto (ORB, Object Request Broker) que permite aos clientes invocar métodos em objetos remotos, em que clientes e servidores podem ser implementados em uma variedade de linguagens de programação. Os principais componentes da arquitetura CORBA estão ilustrados na Figura 8.5.

Essa figura deve ser comparada com a Figura 5.15, para que se note que a arquitetura CORBA contém três componentes adicionais: o adaptador de objeto, o repositório de implementações e o repositório de interfaces.

O CORBA fornece invocações estáticas e dinâmicas. As invocações estáticas são usadas quando a interface remota do objeto CORBA é conhecida no momento da compilação, permitindo o uso de *stubs* de cliente e esqueletos de servidor. Se a interface remota não for conhecida no momento da compilação, a invocação dinâmica deverá ser usada. A maioria dos programadores prefere usar invocação estática, pois ela fornece um modelo de programação mais natural.

Discutiremos, agora, os componentes da arquitetura, deixando as preocupações com a invocação dinâmica por último.

Núcleo ORB • A função do núcleo ORB inclui toda a funcionalidade do módulo de comunicação da Figura 5.15. Além disso, um núcleo ORB fornece uma interface com o seguinte:

- operações que o permitem ser iniciado e parado;
- operações para fazer a conversão entre referências de objeto remoto e *strings*;
- operações para fornecer listas de argumentos para requisições usando invocação dinâmica.

Adaptador de objeto • A função de um *adaptador de objeto* é fazer a ligação entre objetos CORBA com interfaces IDL e as interfaces da linguagem de programação das classes serventes correspondentes. Essa função também inclui a dos módulos de referência remota e despachante na Figura 5.15. Um adaptador de objeto tem as seguintes tarefas:

- Ele cria referências de objeto remoto para objetos CORBA (veja a Seção 8.3.3).
- Ele envia cada RMI, por meio de um esqueleto, para o servente apropriado.
- Ele ativa e desativa serventes.

Um adaptador de objeto fornece para cada objeto CORBA um *nome de objeto* exclusivo, o qual faz parte de sua referência de objeto remoto. O mesmo nome é usado sempre que um objeto é ativado. O nome do objeto pode ser especificado pelo programa aplicativo ou gerado pelo adaptador de objeto. Cada objeto CORBA é registrado em seu adaptador de objeto, o qual mantém uma tabela de objetos remotos que faz o mapeamento dos nomes de objetos CORBA para seus serventes.

Cada adaptador de objeto tem seu próprio nome, o qual faz parte das referências de objeto remoto de todos os objetos CORBA que gerencia. Esse nome pode ser especificado pelo programa aplicativo ou gerado automaticamente.

Adaptador de objeto portátil • O padrão CORBA para adaptador de objetos é chamado de *Portable Object Adapter* (POA). Ele é chamado portátil porque permite que aplicações e serventes sejam executados em ORBs produzidos por diferentes desenvolvedores [Vinoski 1998]. Isso é obtido por meio da padronização das classes de esqueleto e das interações entre o POA e os serventes.

O POA suporta objetos CORBA com dois tipos diferentes de tempos de vida:

- aqueles cujos tempos de vida são restritos ao do processo em que seus serventes são instanciados;
- aqueles cujos tempos de vida podem abranger as instanciações de serventes em vários processos.

Os primeiros têm referências de objeto transientes e os últimos têm referências de objeto persistentes (veja a Seção 8.3.3).

O POA permite que objetos CORBA sejam instanciados de forma transparente e, além disso, separa a criação de objetos CORBA da criação dos *serventes* que implementam esses objetos. Aplicações servidoras, como bancos de dados, com grandes números de objetos CORBA, podem criar serventes sob demanda, apenas quando os objetos são acessados. Neste caso, elas podem usar chaves de banco de dados para os nomes de objeto ou podem usar um único servente para suportar todos esses objetos.

Além disso, é possível especificar políticas para o POA; por exemplo, se ele deve fornecer uma *thread* para cada invocação, se as referências de objeto devem ser persistentes ou transientes e se deve haver um servente separado para cada objeto CORBA. O padrão é que um único servente possa representar todos os objetos CORBA para seu POA.

Note que as implementações de CORBA fornecem interfaces para a funcionalidade do POA e do núcleo ORB por meio de *pseudo-objetos*, os quais recebem esse nome porque não podem ser usados como objetos CORBA normais; por exemplo, eles não podem ser passados como argumentos em RMIs. Contudo, eles podem ter interfaces IDL e são implementados como bibliotecas. O pseudo-objeto POA inclui, por exemplo, um método para ativar um *POAmanager* e outro, *servant_to_reference*, para registrar um objeto CORBA. O pseudo-objeto ORB inclui o método *init*, que deve ser chamado para inicializar o ORB, o método *resolve_initial_references*, que é usado para localizar serviços como o serviço de nomes e o POA raiz, e outros métodos, que possibilitam fazer conversões entre referências de objeto remoto e *strings*.

Esqueletos • As classes de esqueleto são geradas na linguagem do servidor por um compilador de IDL. Como descrito na Seção 5.4.2, as invocações de método remotas são enviadas por meio do esqueleto apropriado para um servente em particular, e o esqueleto desempacota os argumentos nas mensagens de requisição e empacota exceções e resultados nas mensagens de resposta.

Stubs/proxies de cliente • Eles são escritos na linguagem do cliente. A classe de um *proxy* (para linguagens orientadas a objetos) ou um conjunto de procedimentos de *stub* (para linguagens procedurais) é gerada a partir de uma interface IDL por um compilador de IDL para a linguagem do cliente. Como antes, os *stubs/proxies* de cliente empacotam os argumentos nas requisições de invocação e desempacotam exceções e resultados nas respostas.

Repositório de implementações • Um repositório de implementações é responsável por ativar servidores registrados sob demanda e por localizar os servidores que estão correntemente em execução. O nome do adaptador de objeto é usado para se referir aos servidores ao registrá-los e ativá-los.

Um repositório de implementações armazena um mapeamento dos nomes de adaptadores de objeto para nomes de caminho de arquivos contendo implementações de objeto. As implementações de objeto e os nomes de adaptadores de objeto geralmente são registrados no repositório de implementações quando os programas servidores são instalados. Quando as implementações de objeto são ativadas nos servidores, o nome de computador servidor (*hostname*) e o seu número da porta são adicionados no mapeamento:

Entrada do repositório de implementações:

nome do adaptador de objeto	nome de caminho da implementação de objeto	<i>hostname</i> e número da porta do servidor
-----------------------------	--	---

Nem todos os objetos CORBA precisam ser ativados sob demanda. Alguns deles, por exemplo, objetos de *callback* criados pelos clientes, são executados uma vez e deixam de existir quando não são mais necessários. Eles não usam o repositório de implementações.

Geralmente, um repositório de implementações permite que sejam armazenadas informações extras sobre cada servidor, como informações de controle de acesso sobre quem pode ativá-lo ou executar suas operações. É possível replicar informações nos repositórios de implementação para fornecer disponibilidade ou tolerância a falhas.

Repositório de interfaces • A função do repositório de interfaces é fornecer informações sobre as interfaces IDL registradas para clientes e servidores que as exigirem. Para uma interface de determinado tipo, ele pode fornecer os nomes dos métodos e, para cada método, os nomes e tipos dos argumentos e exceções. Assim, o repositório de interfaces adiciona um recurso de reflexão no CORBA. Suponha que um programa cliente receba uma referência remota para um novo objeto CORBA. Se o cliente não tem *proxy* para ele, pode perguntar ao repositório de interfaces sobre os métodos do objeto e os tipos de parâmetro que cada um deles exige.

Quando um compilador de IDL processa uma interface, ele atribui um identificador de tipo para cada tipo de IDL que encontra. Para cada interface registrada nele, o repositório de interfaces fornece um mapeamento entre o identificador de tipo dessa interface e a interface em si. Assim, o identificador de tipo de uma interface é também chamado de *ID de repositório*, pois pode ser usado como uma chave para as interfaces IDL registradas no repositório de interfaces.

Toda referência de objeto remoto CORBA inclui uma entrada (*slot*) que contém o identificador de tipo de sua interface, permitindo que os clientes que o possuem perguntem sobre seu tipo no repositório de interfaces. Essas aplicações que usam invocação estática (normal) com *proxies* clientes e esqueletos de IDL não exigem repositório de interfaces. Nem todos os ORBs fornecem repositório de interfaces.

Interface de invocação dinâmica • Conforme sugerido na Seção 5.5, em algumas aplicações pode ser necessário construir um programa cliente sem conhecer todas as classes *proxy* de que ele precisará no futuro. Por exemplo, talvez um navegador de objeto precise exibir informações sobre todos os objetos CORBA disponíveis nos vários servidores em um sistema distribuído. Não é exequível que tal programa inclua *proxies* para todos esses objetos, particularmente porque, à medida que o tempo passa, novos objetos podem ser adicionados no sistema. O CORBA não permite baixar classes de *proxies* por *download* em tempo de execução, como RMI Java. A interface de invocação dinâmica é a alternativa do CORBA.

A interface de invocação dinâmica permite que os clientes façam invocações dinâmicas em objetos remotos CORBA. Ela é usada quando não é possível empregar *proxies*. O cliente pode obter do repositório de interfaces as informações necessárias sobre os métodos disponíveis para determinado objeto CORBA e pode usar essas informações para construir uma invocação com argumentos convenientes e enviá-la para o servidor.

Esqueletos dinâmicos • Novamente, conforme explicado na Seção 5.5, pode ser necessário adicionar a um servidor um objeto CORBA cuja interface era desconhecida quando o servidor foi compilado. Se um servidor usa esqueletos dinâmicos, ele pode aceitar invocações na interface de um objeto CORBA para as quais não possui esqueleto. Quando um esqueleto dinâmico recebe uma invocação, ele inspeciona o conteúdo da requisição para descobrir seu objeto de destino, o método a ser invocado e os argumentos. Então, ele ativa o destino.

Código legado • O termo *código legado* se refere a código existente que não foi projetado tendo-se em mente os objetos distribuídos. Um código legado pode ser transformado em um objeto CORBA definindo-se uma interface IDL para ele e fornecendo-se a implementação de um adaptador de objeto apropriado e os esqueletos necessários.

8.3.3 Referências de objeto remoto CORBA

O CORBA especifica um formato para referências de objeto remoto conveniente para uso, seja o objeto remoto invocado por um repositório de implementações ou não. As referências que usam esse formato são chamadas de *referências de objeto interoperáveis* (*IORs*, *Interoperable Object References*). A figura a seguir é baseada em Henning [1998], que contém um relato mais detalhado das IORs:

Formato IOR

ID de tipo de interface IDL	Protocolo e detalhes do endereço			Chave de objeto	
identificador de repositório de interfaces	IIOP	nome de domínio do <i>host</i>	número da porta	nome do adaptador	nome do objeto

Seguindo a sequência dos vários campos:

- O primeiro campo de uma IOR especifica o identificador de tipo da interface IDL do objeto CORBA. Note que, se o ORB tiver um repositório de interfaces, esse nome de tipo também será o identificador do repositório de interfaces da interface IDL, o qual permite recuperar a definição de IDL da interface em tempo de execução.
- O segundo campo especifica o protocolo de transporte e os detalhes exigidos por esse protocolo de transporte específico para identificar o servidor. Em particular, o protocolo IIOP (Internet Inter-ORB) usa TCP, no qual o endereço do servidor

consiste em um nome de domínio de computador (*host*) e um número de porta [OMG 2004a].

- O terceiro campo é usado pelo ORB para identificar um objeto CORBA. Ele consiste no nome de um adaptador de objeto no servidor e no nome de um objeto CORBA, especificado pelo adaptador de objeto.

As *IORs transientes* dos objetos CORBA duram apenas tanto quanto o processo que contém esses objetos, enquanto as *IORs persistentes* duram entre as invocações dos objetos CORBA. Uma IOR transiente contém os detalhes do endereço do servidor que contém o objeto CORBA, enquanto uma IOR persistente contém os detalhes do endereço do repositório de implementações no qual está registrada. Nos dois casos, o cliente ORB envia a mensagem de requisição para o servidor cujos detalhes do endereço são dados na IOR. Aqui está como a IOR é usada para localizar o servente que representa o objeto CORBA nos dois casos:

IORs transientes: o núcleo ORB servidor recebe a mensagem de requisição contendo o nome do adaptador de objeto e o nome do objeto do destino. Ele usa o nome do adaptador de objeto para localizar o adaptador de objeto, o qual utiliza o nome do objeto para localizar o servente.

IORs persistentes: um repositório de implementações recebe a requisição. Ele extrai o nome do adaptador de objeto da IOR presente na requisição. Desde que o nome do adaptador de objeto esteja em sua tabela, ele tenta, se necessário, ativar o objeto CORBA no endereço de *host* especificado em sua tabela. Uma vez que o objeto CORBA tenha sido ativado, o repositório de implementações retorna seus detalhes de endereço para o cliente ORB, o qual os utiliza como destino das mensagens de requisição RMI, que incluem o nome do adaptador de objeto e o nome do objeto. Isso permite que o núcleo ORB servidor localize o adaptador de objeto, o qual, como antes, utiliza o nome do objeto para localizar o servente.

O segundo campo de uma IOR pode ser repetido, de modo a especificar o nome de domínio do *host* e o número da porta de mais de um destino, para permitir que um objeto ou um repositório de implementações seja replicado em vários locais diferentes.

A mensagem de resposta no protocolo de requisição-resposta inclui informações de cabeçalho que permitem a execução do procedimento das IORs persistentes. Em particular, ela inclui uma entrada de *status* que pode indicar se a requisição deve ser encaminhada para um servidor diferente, no caso em que o corpo da resposta inclui uma IOR que contém o endereço do servidor do objeto recentemente ativado.

8.3.4 Serviços CORBA

O CORBA inclui especificações de serviços que podem ser exigidos por objetos distribuídos. Em particular, o serviço de nomes (*Naming Service*) é uma adição essencial em qualquer ORB, como veremos em nosso exemplo de programação na Seção 8.3.5. Um índice para documentação sobre todos os serviços pode ser encontrado no *site* do OMG, no endereço [www.omg.org]. Muitos dos serviços CORBA estão descritos em Orfali *et al.* [1996, 1997]. Um resumo dos principais serviços CORBA aparece na Figura 8.6. Mais detalhes sobre esses serviços podem ser encontrados no *site* que acompanha o livro [www.cdk5.net/corba] (em inglês).

<i>Serviço CORBA</i>	<i>Função</i>	<i>Mais detalhes</i>
<i>Serviço de nomes</i>	Oferece atribuição de nomes no CORBA, em particular, realiza o mapeamento de nomes em referências de objeto remoto dentro de determinado contexto de atribuição de nomes (veja o Capítulo 9).	[OMG 2004b]
<i>Serviço de negócio</i>	Enquanto o serviço de nomes permite localizar objetos pelo nome, o <i>Trading Service</i> (serviço de negócio) permite localizá-los pelo atributo; isto é, ele é um serviço de diretório. O banco de dados subjacente gerencia um mapeamento dos tipos de serviço e seus atributos associados para referências de objeto remoto.	[OMG 2000a, Henning e Vinoski 1999]
<i>Serviço de evento</i>	Permite que objetos de interesse transmitam notificações para os assinantes, usando invocações de método remoto CORBA (veja o Capítulo 6 para mais sobre serviços de evento em geral).	[Farley 1998, OMG 2004c]
<i>Serviço de notificação</i>	Amplia o serviço de evento, com os recursos adicionais incluindo a capacidade de definir filtros expressando eventos de interesse e de definir propriedades de confiabilidade e ordenação do canal de eventos subjacente.	[OMG 2004d]
<i>Serviço de segurança</i>	Suporta diversos mecanismos de segurança, incluindo autenticação, controle de acesso, comunicação segura, auditoria e não repúdio (veja o Capítulo 11).	[Blakely 1999, Baker 1997, OMG 2002b]
<i>Serviço de transação</i>	Suporta a criação de transações planas ou aninhadas (conforme definidas nos Capítulos 16 e 17).	[OMG 2003]
<i>Serviço de controle de concorrência</i>	Usa travas para aplicar controle de concorrência no acesso de objetos CORBA (pode ser usado por meio do serviço de transação ou como um serviço independente).	[OMG 2000b]
<i>Serviço de estado persistente</i>	Oferece um repositório de objetos persistentes para o CORBA, usado para salvar e restaurar o estado de objetos CORBA (as implementações são recuperadas a partir do repositório de implementações).	[OMG 2002d]
<i>Serviço de ciclo de vida</i>	Define convenções para criar, excluir, copiar e mover objetos CORBA; por exemplo, como usar fábricas para criar objetos.	[OMG 2002e]

Figura 8.6 Serviços CORBA.

```

public interface ShapeListOperations {
    Shape newShape(GraphicalObject g) throws ShapeListPackage.FullException;
    Shape[] allShapes();
    int getVersion();
}

public interface ShapeList extends ShapeListOperations, org.omg.CORBA.Object,
    org.omg.CORBA.portable.IDLEntity {}

```

Figura 8.7 Interfaces Java geradas pelo *idlj* a partir da interface CORBA *ShapeList*.

8.3.5 Exemplo de cliente e servidor CORBA

Esta seção descreve, em linhas gerais, os passos necessários para produzir programas cliente e servidor que usam as interfaces IDL *Shape* e *ShapeList* mostradas na Figura 8.2. A seguir, apresentaremos uma discussão sobre *callbacks* no CORBA. Usamos Java como linguagem de programação tanto para o cliente quanto para o servidor, mas a estratégia é semelhante para outras linguagens. O compilador de interface *idlj* pode ser aplicado nas interfaces CORBA para gerar os seguintes itens:

- As interfaces Java equivalentes – duas por interface IDL. O nome da primeira interface Java termina com *Operations* – essa interface define apenas as operações na interface IDL. A segunda interface Java tem o mesmo nome da interface IDL e implementa as operações da primeira interface, assim como as de uma interface conveniente para um objeto CORBA. Por exemplo, a interface IDL *ShapeList* resulta em duas interfaces Java *ShapeListOperations* e *ShapeList*, como mostrado na Figura 8.7.
- Os esqueletos de servidor para cada interface *idl*. Os nomes das classes de esqueleto terminam com *POA* – por exemplo, *ShapeListPOA*.
- As classes *proxy* ou *stubs* de cliente, uma para cada interface IDL. Os nomes dessas classes terminam com *Stub* – por exemplo, *_ShapeListStub*.
- Uma classe Java para corresponder a cada uma das *structs* definidas com as interfaces IDL. Em nosso exemplo, são geradas as classes *Rectangle* e *GraphicalObject*. Cada uma dessas classes contém uma declaração de uma variável de instância para cada campo na *struct* correspondente e dois construtores, mas nenhum outro método.
- Classes chamadas auxiliares (*helpers*) e portadoras (*holders*), uma para cada um dos tipos definidos na interface IDL. Uma classe auxiliar contém o método *narrow*, que é usado para converter determinada referência de objeto para a classe a que pertence e que está mais abaixo na hierarquia de classes. Por exemplo, o método *narrow* em *ShapeHelper* é convertido para a classe *Shape*. As classes portadoras lidam com argumentos *out* e *inout*, os quais não podem ser mapeados diretamente para Java. Veja no Exercício 8.9 um exemplo do uso de classes portadoras.

Programa servidor • O programa servidor deve conter implementações de uma ou mais interfaces IDL. Para um servidor escrito em uma linguagem orientada a objetos, como Java ou C++, essas interfaces são implementadas como classes serventes. Os objetos CORBA são instâncias de classes serventes.

```

import org.omg.CORBA.*;
import org.omg.PortableServer.POA;
class ShapeListServant extends ShapeListPOA {
    private POA theRootpoa;
    private Shape theList[];
    private int version;
    private static int n=0;
    public ShapeListServant(POA rootpoa){
        theRootpoa = rootpoa;
        // inicializa as outras variáveis de instância
    }
    public Shape newShape(GraphicalObject g) throws ShapeListPackage.FullException {    1
        version++;
        Shape s = null;
        ShapeServant shapeRef = new ShapeServant( g, version);
        try {
            org.omg.CORBA.Object ref = theRootpoa.servant_to_reference(shapeRef);    2
            s = ShapeHelper.narrow(ref);
        } catch (Exception e) { }
        if(n >=100) throw new ShapeListPackage.FullException( );
        theList[n++] = s;
        return s;
    }
    public Shape[] allShapes() { ... }
    public int getVersion() { ... }
}

```

Figura 8.8 Classe *ShapeListServant* do programa servidor em Java da interface CORBA *ShapeList*.

Quando um servidor cria uma instância de uma classe servente, ele deve registrá-la no POA, o qual transforma a instância em um objeto CORBA e fornece a ele uma referência de objeto remoto. Se isso não for feito, o objeto CORBA não poderá receber invocações remotas. Os leitores que estudaram cuidadosamente o Capítulo 5 poderão perceber que o registro do objeto no POA o faz ser registrado no equivalente CORBA da tabela de objetos remotos.

Em nosso exemplo, o servidor contém implementações das interfaces *Shape* e *ShapeList* na forma de duas classes serventes, junto a uma classe servidora que contém uma seção *initialization* (veja a Seção 5.4.2) em seu método *main*.

As classes serventes: cada classe servente estende a classe esqueleto correspondente e implementa os métodos de uma interface IDL usando as assinaturas de método definidas na interface Java equivalente. A classe servente que implementa a interface *ShapeList* é chamada de *ShapeListServant*, embora qualquer outro nome pudesse ser escolhido. Seu esboço aparece na Figura 8.8. Considere o método *newShape*, na linha 1, que é um método de fábrica, pois cria objetos *Shape*. Para transformar um objeto *Shape* em um objeto CORBA, ele é registrado no POA por intermédio de seu método *servant_to_reference*, como mostrado na linha 2, que faz uso da referência ao POA raiz, que foi passado por meio do construtor quando a classe servente foi criada. Versões completas da interface IDL e das classes cliente e servidor deste exemplo estão disponíveis na página do livro em www.grupoa.com.br ou em www.cdk5.net/corba (em inglês).

```

import org.omg.CosNaming.*;
import org.omg.CosNaming.NamingContextPackage.*;
import org.omg.CORBA.*;
import org.omg.PortableServer.*;
public class ShapeListServer {
    public static void main(String args[] ) {
        try{
            ORB orb = ORB.init(args, null);
            POA rootpoa = POAHelper.narrow(orb.resolve_initial_references("RootPOA"));
            rootpoa.the_POAManager().activate();
            ShapeListServant SLSRef = new ShapeListServant(rootpoa);
            org.omg.CORBA.Object ref = rootpoa.servant_to_reference(SLSRef);
            ShapeList SLRef = ShapeListHelper.narrow(ref);
            org.omg.CORBA.Object objRef = orb.resolve_initial_references("NameService");
            NamingContext ncRef = NamingContextHelper.narrow(objRef);
            NameComponent nc = new NameComponent("ShapeList", "");
            NameComponent path[] = {nc};
            ncRef.rebind(path, SLRef);
            orb.run();
        } catch (Exception e) { ... }
    }
}

```

Figura 8.9 Classe Java *ShapeListServer*.

O servidor: o método *main* na classe servidora *ShapeListServer* aparece na Figura 8.9. Primeiramente, ele cria e inicializa o ORB (linha 1) e, então, obtém uma referência para o POA raiz e ativa o POAManager (linhas 2 e 3). Em seguida, ele cria uma instância de *ShapeListServant*, que é apenas um objeto Java (linha 4) e, ao fazer isso, passa uma referência para o POA raiz. Então, ele o transforma em um objeto CORBA, registrando-o no POA (linha 5). Depois disso, ele registra o servidor no serviço de nomes. Então, ele espera receber requisições de clientes (linha 10).

Um servidor que usa o serviço de nomes, primeiro obtém um contexto de atribuição de nomes raiz usando *NamingContextHelper* (linha 6). O contexto de atribuição de nomes define o escopo dentro do qual um conjunto de nomes se aplica (cada um dos nomes deve ser exclusivo dentro de um contexto). Depois, o servidor faz um *NameComponent* (linha 7), sendo *NameComponent* um objeto que representa um nome no CORBA. Isso tem duas partes: um *nome* e um *tipo*. O campo de *tipo* é puramente descritivo (o campo é usado pelas aplicações e não é interpretado pelo serviço de nomes). Os nomes podem ser compostos e representados por um *caminho* para o objeto no grafo de nomes. Neste exemplo, não são usados nomes compostos; em vez disso, o caminho consiste em um único nome, definido na linha 8. Finalmente, o servidor usa o método *rebind* do serviço de nomes (linha 9), o qual registra o par nome e referência de objeto remoto no contexto apropriado. Os clientes executam os mesmos passos, mas utilizam o método *resolve*, como mostrado na Figura 8.10, linha 2.

O programa cliente • Um exemplo de programa cliente aparece na Figura 8.10. Ele cria e inicializa um ORB (linha 1) e, depois, entra em contato com o serviço de nomes para


```

import org.omg.CosNaming.*;
import org.omg.CosNaming.NamingContextPackage.*;
import org.omg.CORBA.*;
public class ShapeListClient{
    public static void main(String args[] ) {
        try{
            ORB orb = ORB.init(args, null);
            org.omg.CORBA.Object objRef =
            orb.resolve_initial_references("NameService");
            NamingContext ncRef = NamingContextHelper.narrow(objRef);
            NameComponent nc = new NameComponent("ShapeList", "");
            NameComponent path [ ] = { nc };
            ShapeList shapeListRef =
            ShapeListHelper.narrow(ncRef.resolve(path));
            Shape[ ] sList = shapeListRef.allShapes( );
            GraphicalObject g = sList[0].getAllState( );
        } catch(org.omg.CORBA.SystemException e) {...}
    }
}

```

Figura 8.10 Programa cliente em Java para as interfaces CORBA *Shape* e *ShapeList*.

obter uma referência para o objeto remoto *ShapeList*, usando seu método *resolve* (linha 2). Depois disso, ele invoca seu método *allShapes* (linha 3) para obter uma sequência de referências de objeto remoto para todos os objetos *Shape* correntemente mantidos no servidor. Em seguida, ele invoca o método *getAllState* (linha 4), fornecendo como argumento a primeira referência de objeto remoto da sequência retornada; o resultado é fornecido como uma instância da classe *GraphicalObject*.

O método *getAllState* parece contradizer nossa afirmação anterior de que objetos não podem ser passados por valor no CORBA, pois tanto o cliente como o servidor transacionam em instâncias da classe *GraphicalObject*. Entretanto, não há contradição: o objeto CORBA retorna uma *struct*, e os clientes que estiverem usando uma linguagem diferente poderão vê-la de forma diferente. Por exemplo, na linguagem C++, o cliente a veria como uma *struct*. Mesmo em Java, a classe *GraphicalObject* gerada é mais parecida com uma *struct*, pois não tem métodos.

Os programas clientes sempre devem capturar exceções CORBA *SystemExceptions*, as quais relatam erros decorrentes da distribuição (veja a linha 5). Os programas clientes também devem capturar as exceções definidas na interface IDL, como a *FullException* disparada pelo método *newShape*.

Esse exemplo ilustra o uso da operação *narrow*: a operação *resolve* do serviço de nomes retorna um valor de tipo *Object*; esse tipo é reduzido para estar de acordo com o tipo em particular exigido – (*ShapeList*).

Callbacks • *Callbacks* podem ser implementadas no CORBA de uma maneira semelhante àquela descrita para RMI Java, na Seção 5.5.1. Por exemplo, a interface *WhiteboardCallback* pode ser definida como segue:

```

interface WhiteboardCallback {
    oneway void callback(in int version);
};

```

Essa interface é implementada como um objeto CORBA pelo cliente, permitindo que o servidor envie ao cliente um número de versão quando novos objetos são adicionados. No entanto, antes que o servidor possa fazer isso, o cliente precisa informá-lo da referência de objeto remoto de seu objeto. Para tornar isso possível, a interface *ShapeList* exige métodos adicionais, como *register* e *deregister*, conforme segue:

```
int register(in WhiteboardCallback callback);  
void deregister(in int callbackId);
```

Após um cliente ter obtido uma referência para o objeto *ShapeList* e criado uma instância de *WhiteboardCallback*, ele usa o método *register* de *ShapeList* para informar o servidor de que está interessado em receber *callbacks*. O objeto *ShapeList* no servidor é responsável por manter uma lista de clientes interessados e notificar a todos eles sempre que seu número de versão aumentar, quando um novo objeto for adicionado. O método *callback* é declarado como *oneway* para que o servidor possa usar chamadas assíncronas para evitar atraso ao notificar cada cliente.

8.4 De objetos a componentes

O *middleware* de objeto distribuído tem sido expressivamente implementado em uma grande variedade de aplicações, incluindo as áreas apresentadas no Capítulo 1: finanças e comércio, assistência médica, educação, transporte e logística e assim por diante. As técnicas incorporadas no CORBA e as plataformas relacionadas se mostraram bem-sucedidas em enfrentar muitos dos principais problemas associados à programação distribuída, especialmente os relacionados a solucionar a heterogeneidade e a permitir a portabilidade e a operação conjunta de *software* de sistemas distribuídos. A variedade de serviços que acompanha essas plataformas também estimula o desenvolvimento de *software* seguro e confiável.

No entanto, foram identificadas várias deficiências. Isso levou ao surgimento do que definimos como estratégias baseadas em componentes, como uma evolução natural da computação de objeto distribuído. Esta seção apresenta essa transição, discutindo os requisitos que levaram às estratégias baseadas em componentes e fornece uma definição de componentes, antes de examinar com mais profundidade os estilos de estratégias baseadas em componentes adotadas nos sistemas distribuídos. A seguir, a Seção 8.5, apresentará dois estudos de caso de tecnologias de componente contrastantes: Enterprise JavaBeans e Fractal.

Problemas do middleware orientado a objetos • Conforme mencionado anteriormente, as estratégias baseadas em componentes surgiram para enfrentar os problemas identificados na computação de objeto distribuído. Os problemas foram listados na Seção 8.1 e serão discutidos com mais detalhes a seguir.

Dependências implícitas: um objeto distribuído oferece um *contrato* para o mundo externo em termos da interface (ou interfaces) que apresenta para o ambiente distribuído. O contrato representa um acordo de compromisso entre o provedor do objeto e os usuários desse objeto, em termos de seu comportamento esperado. Frequentemente, supõe-se que essas interfaces forneçam um contrato completo, no sentido de distribuir e usar esse objeto. Contudo, esse não é o caso. O problema surge do fato de o comportamento interno (encapsulado) de um objeto ser oculto. Por exemplo, um objeto pode se comunicar com outro objeto ou com o serviço de sistema distribuído associado por meio de uma

invocação de método remoto ou de outro paradigma de comunicação. Se olharmos os programas servidor e cliente CORBA mostrados nas Figuras 8.9 e 8.10, respectivamente, veremos que o servidor emite uma *callback* para o cliente e isso não fica aparente na interface definida no servidor. Além disso, embora tanto o cliente como o servidor se comuniquem com o serviço de nomes, novamente isso não é visível a partir da observação externa desse objeto (conforme oferecida pelas interfaces).

Mais geralmente, determinado objeto pode fazer chamadas arbitrárias para outros objetos em nível de aplicação ou para serviços do sistema distribuído que oferecem atribuição de nomes, persistência, controle de concorrência, transações, segurança, etc., e isso não é capturado na visão externa da configuração. As dependências implícitas na configuração distribuída tornam difícil garantir a composição segura de uma configuração, substituir um objeto por outro e a implementação de um elemento em particular em uma configuração distribuída por parte de desenvolvedores externos.

Requisito: a partir disso, há a necessidade clara de não apenas especificar as interfaces oferecidas por um objeto, mas também as dependências desse objeto em relação a outros na configuração distribuída.

Interação com o middleware: apesar dos objetivos da transparência, é claro que, ao usar *middleware* de objeto distribuído, o programador fica exposto a muitos detalhes de nível relativamente baixo, associados à arquitetura do *middleware*. Novamente, o exemplo de cliente-servidor mostrado nas Figuras 8.9 e 8.10 fornecem uma ilustração disso. Apesar de ser uma aplicação muito simples, existem muitas chamadas relacionadas ao CORBA que são absolutamente fundamentais para o funcionamento da aplicação. Isso inclui chamadas associadas aos nomes (conforme mencionado anteriormente), ao POA e ao núcleo ORB. Em exemplos mais complexos, isso poderia incluir código arbitrariamente sofisticado em termos da criação e do gerenciamento de referências de objeto, gerenciamento de ciclos de vida de objetos, políticas de ativação e passivação, gerenciamento de estado persistente e políticas para mapeamentos nos recursos da plataforma subjacente, como as *threads*. Tudo isso pode se tornar, muito rapidamente, um desvio do principal objetivo do código, que é criar um aplicativo. Isso fica muito evidente no exemplo citado anteriormente, em que o código realmente relacionado ao aplicativo de quadro negro (*whiteboard*) é mínimo e intercalado com código relacionado às preocupações com os sistemas distribuídos.

Requisito: há uma necessidade evidente de simplificar a programação de aplicações distribuídas para apresentar uma clara separação de preocupações entre código relacionado à operação em uma estrutura de *middleware* e código associado à aplicação, e de permitir que o programador se concentre exclusivamente neste último.

Falta de separação de aspectos relacionados à distribuição: os programadores que usam *middleware* de objeto distribuído também precisam lidar explicitamente com preocupações não funcionais relacionadas a problemas como segurança, transações, coordenação e replicação. Em tecnologias como CORBA e RMI, isso é conseguido pela inserção de chamadas apropriadas nos serviços do sistema distribuído associados dentro dos objetos. Isso tem duas repercussões:

- Os programadores precisam ter profundo conhecimento dos detalhes completos de todos os serviços do sistema distribuído associados,
- A implementação de determinado objeto conterá código de aplicação junto a chamadas para serviços do sistema distribuído e para as interfaces do *middleware* sub-

jacente (conforme mencionado anteriormente). O emaranhado de possibilidades aumenta ainda mais a complexidade da programação de sistemas distribuídos.

Requisito: a separação de aspectos mencionada anteriormente também deve abranger o tratamento dos diversos serviços do sistema distribuído e, quando possível, as complexidades do tratamento de tais serviços devem ficar ocultas do programador.

Falta de suporte para distribuição: embora tecnologias como CORBA e RMI Java tornem possível desenvolver configurações distribuídas arbitrárias de objetos, não há suporte para a distribuição dessas configurações. Em vez disso, os objetos precisam ser distribuídos manualmente em máquinas individuais. Esse processo pode ser cansativo e propenso a erros, particularmente em distribuições em larga escala, que consistem em muitos objetos, em uma arquitetura física com um grande número de nós computacionais (potencialmente heterogêneos). Além de ser necessário posicioná-los fisicamente, os objetos também precisam ser ativados, e vínculos apropriados para outros objetos precisam ser criados. Por causa dessa falta de suporte para distribuição, inevitavelmente os desenvolvedores contam com estratégias de distribuição *ad hoc*, as quais não são portáteis para outros ambientes.

Requisito: as plataformas de *middleware* devem fornecer suporte intrínseco para a distribuição, para que o *software* distribuído possa ser instalado e implantado da mesma maneira que um *software* para uma única máquina, com as complexidades da distribuição oculta do usuário.

Esses quatro requisitos levaram ao surgimento de estratégias baseadas em componentes para o desenvolvimento de sistemas distribuídos, junto ao surgimento de *middleware baseado em componentes*, incluindo o estilo de *middleware* chamado de *servidores de aplicação*.

Note que, embora as estratégias baseadas em componentes só tenham ganhado importância nos últimos anos, suas raízes remontam aos primeiros projetos que tratavam de reconfiguração em sistemas distribuídos (como o Projeto Conic do Imperial College London [Magee e Sloman 1989]).

Essência dos componentes • Para os propósitos desta discussão, adotamos a definição de componentes fornecida por Szyperski em seu livro sobre *software* de componentes [Szyperski 2002]:

Componentes: um componente de *software* é uma unidade de composição com interfaces especificadas por contratos e somente dependências contextuais explícitas.

Nessa definição clássica, a palavra “somente” se refere ao fato de que quaisquer dependências contextuais devem ser explícitas – isto é, não estão presentes quaisquer dependências implícitas.

Os componentes de *software* são como os objetos distribuídos, no sentido de serem unidades de composição encapsuladas, mas determinado componente especifica suas interfaces fornecidas para o mundo externo e suas dependências de outros componentes no ambiente distribuído. As dependências também são representadas como interfaces. Mais especificamente, um componente é especificado em termos de um *contrato*, o qual inclui:

- um conjunto de *interfaces fornecidas* – isto é, as interfaces que o componente oferece como serviços para outros componentes;

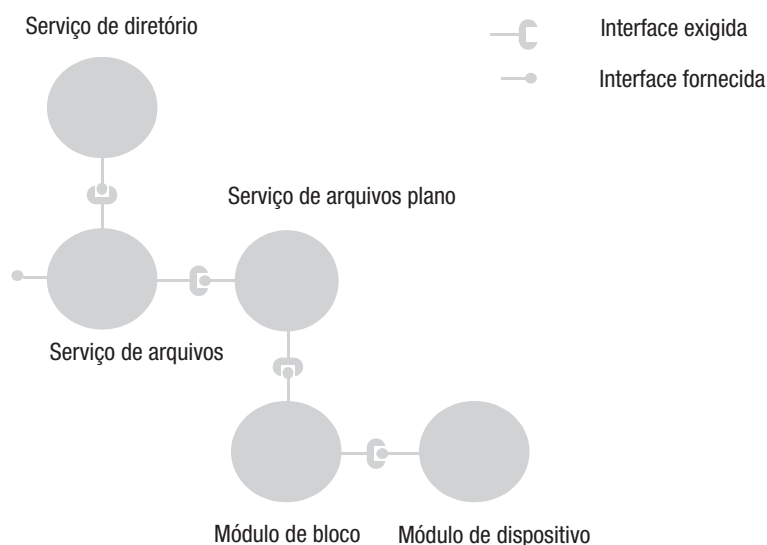


Figura 8.11 Um exemplo de arquitetura de software.

- um conjunto de *interfaces exigidas* – isto é, as dependências que esse componente tem em termos de outros componentes que devem estar presentes e conectados a ele para que ele funcione corretamente.

Em determinada configuração de componente, toda interface exigida deve estar ligada a uma interface fornecida de outro componente. Isso também é chamado de *arquitetura de software*, consistindo em componentes, interfaces e conexões entre interfaces. Podemos ver um exemplo dessa configuração na Figura 8.11. Esse exemplo mostra a arquitetura de um sistema de arquivos simples fornecendo uma interface para outros usuários e, por sua vez, exigindo conexão com um componente de serviço de diretório e um componente de serviço de arquivo plano (*flat file service*). A figura também mostra conexões adicionais para módulos de bloco e de dispositivo, capturando a arquitetura global desse sistema de arquivos em particular. (Investigaremos as arquiteturas reais de sistemas de arquivos distribuídos no Capítulo 12.)

As interfaces podem ter diferentes estilos. Em particular, muitas estratégias baseadas em componentes oferecem dois estilos de interfaces:

- interfaces que suportam invocação de método remoto, como em CORBA e RMI Java;
- interfaces que suportam eventos distribuídos (conforme discutido no Capítulo 6).

Veremos exemplos dos dois estilos de interface quando estudarmos o Enterprise JavaBeans, na Seção 8.5.1.

A programação em sistemas baseados em componentes se preocupa com o desenvolvimento de componentes e com sua *composição*. O objetivo é suportar um estilo de desenvolvimento de *software* que se compare ao desenvolvimento de *hardware* no uso de componentes padrão e compô-los para desenvolver serviços mais sofisticados: uma mudança de desenvolvimento de *software* para montagem de *software*. Portanto,

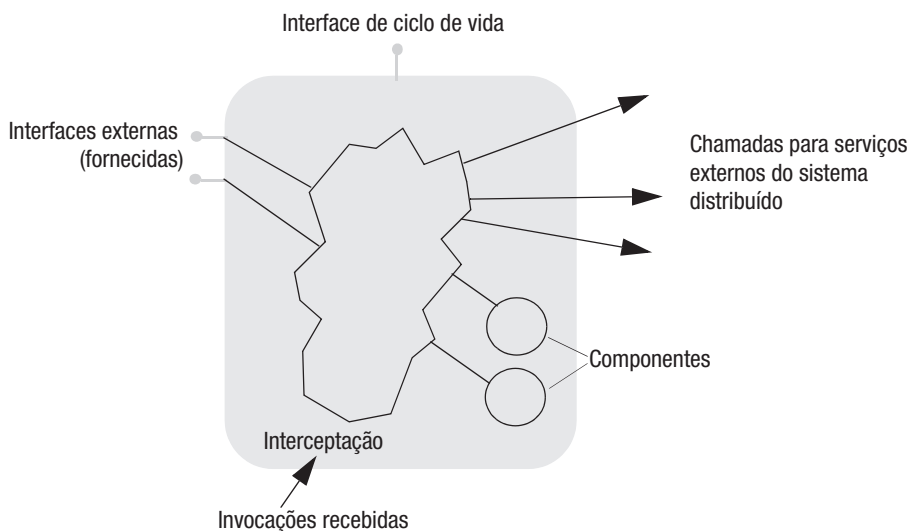


Figura 8.12 A estrutura de um contêiner.

essa estratégia suporta desenvolvimento de componentes de *software* terceirizado e torna mais fácil adaptar configurações de sistema em tempo de execução, substituindo um componente por outro precisamente correspondente em termos das interfaces fornecidas e exigidas.

Note que os defensores das estratégias baseadas em componentes dão ênfase significativa a esse uso de composição e veem isso como a estratégia mais limpa para a construção de sistemas de *software* complexos. Em particular, eles defendem a composição em detrimento da herança, vendo a herança como a criação de formas adicionais de dependência implícita (desta vez, entre classes). Isso pode levar a questões como o problema da classe base frágil, em que alterações em uma classe base podem ter repercussões imprevistas nos objetos que estão mais abaixo na hierarquia de herança [Szyper-ski 2002].

Até aqui, tratamos do primeiro requisito destacado anteriormente (em termos de tornar todas as dependências explícitas), mas não dos três seguintes, que se referem à simplificação do desenvolvimento e da implementação de aplicações distribuídas. Esse nível adicional de suporte vai se tornar aparente quando examinarmos como as estratégias baseadas em componentes têm evoluído na comunidade de sistemas distribuídos.

Componentes e sistemas distribuídos • Diversas tecnologias de *middleware* baseado em componentes vêm surgindo, incluindo o Enterprise JavaBeans (discutido na Seção 8.5.1, a seguir) e o CCM (CORBA Component Model – modelo de componente CORBA) [Wang *et al.* 2001], uma evolução do CORBA de uma plataforma baseada em objetos para uma plataforma baseada em componentes. O *middleware* baseado em componentes complementa a filosofia adotada anteriormente, mas adiciona suporte significativo para o desenvolvimento e a implementação de sistemas distribuídos, conforme discutido a seguir.

<i>Tecnologia</i>	<i>Desenvolvida por</i>	<i>Mais detalhes</i>
<i>WebSphere Application Server</i>	IBM	[www.ibm.com]
<i>Enterprise JavaBeans</i>	SUN	[java.sun.com XII]
<i>Spring Framework</i>	SpringSource (uma divisão da VMware)	[www.springsource.org]
<i>JBoss</i>	JBoss Communit	[www.jboss.org]
<i>CORBA Component Model</i>	OMG	[Wang <i>et al.</i> 2001]
<i>JOonAS</i>	OW2 Consortium	[jonas.ow2.org]
<i>GlassFish</i>	SUN	[glassfish.dev.java.net]

Figura 8.13 Servidores de aplicação.

Contêineres: a noção de *contêineres* é absolutamente fundamental para o *middleware* baseado em componentes. Os contêineres suportam um padrão comum, frequentemente encontrado em aplicações distribuídas, que consiste em:

- um cliente de *front-end* (talvez baseado na Web);
- um contêiner contendo um ou mais componentes que implementam a lógica da aplicação ou do negócio;
- serviços de sistema que gerenciam os dados associados no armazenamento persistente.

(Isso é análogo ao modelo de três camadas físicas, descrito na Seção 2.3.2.)

As tarefas do contêiner são: fornecer um ambiente hospedeiro no lado do servidor *gerenciado* para os componentes e fornecer a separação de aspectos, mencionada anteriormente, na qual os componentes tratam das necessidades da aplicação e o contêiner trata de problemas dos sistemas distribuídos e do *middleware*, garantindo a obtenção de propriedades não funcionais. A estrutura global de um contêiner está apresentada na Figura 8.12. Ela mostra vários componentes encapsulados dentro de um contêiner; o contêiner não fornece acesso direto aos componentes, mas intercepta as invocações recebidas e, então, executa as ações apropriadas para garantir que as propriedades desejadas da aplicação distribuída sejam mantidas. Se pegarmos o caso do CORBA, por exemplo, isso incluiria:

- gerenciar a interação com o núcleo ORB e a funcionalidade do POA subjacentes e ocultá-las totalmente dos desenvolvedores de aplicações;
- gerenciar as chamadas para os serviços do sistema distribuído apropriados, inclusive serviços de segurança e transação, para fornecer as propriedades não funcionais exigidas da aplicação – novamente, de forma transparente para o programador.

Considerados em conjunto, isso pode simplificar significativamente o desenvolvimento de aplicações distribuídas, permitindo que o desenvolvedor de componente se concentre exclusivamente nas preocupações em nível de aplicação. Por exemplo, com uma estratégia de contêiner, todas as chamadas relacionadas ao POA apresentadas nas Figuras 8.8 e 8.9 seriam feitas pelo contêiner e não pelo componente. Analogamente, por intermédio do mecanismo de interceptação, o contêiner pode fazer uma sequência de chamadas potencialmente complexas para serviços apropriados do sistema distribuído, para implementar as propriedades não funcionais exigidas. Como ilustração deste último

ponto, considere a implementação de uma política de gerenciamento simples para tratar do acesso concorrente a um componente. Isso pode ser implementado de maneira transparente para o componente, interceptando-se a invocação recebida na interface externa, adquirindo-se uma trava associada ao componente subjacente e, então, fazendo-se a chamada na operação, no próprio componente, garantindo que a trava seja liberada quando a invocação terminar (vamos falar mais sobre travas na Seção 16.4, mas, por enquanto, um entendimento geral é suficiente).

O *middleware* que suporta o padrão contêiner e a separação de aspectos acarretada por esse padrão é conhecido como *servidor de aplicação*. Esse estilo de programação distribuída está sendo amplamente utilizado no setor. Atualmente, existem diversos servidores de aplicativo; um resumo das principais estratégias está na Figura 8.13. A Seção 8.5.1 examinará a especificação Enterprise JavaBeans como um exemplo de servidor de aplicação.

Suporte para distribuição: o *middleware* baseado em componentes fornece suporte para a distribuição de *configurações* de componente; as versões de *software* são empacotadas como arquiteturas de *software* (componentes e suas interconexões), junto a *descritores de distribuição* que descrevem completamente como a configuração deve ser implantada em um ambiente distribuído.

Note que os componentes são distribuídos em contêineres, sendo que os descritores de distribuição são interpretados pelos contêineres para estabelecer as políticas exigidas pelo *middleware* subjacente e pelos serviços do sistema distribuído. Portanto, determinado contêiner inclui vários componentes que exigem a mesma configuração em termos de suporte para sistema distribuído.

Normalmente, os descritores de distribuição são escritos em XML e incluem informações suficientes para garantir que:

- os componentes sejam corretamente conectados, usando os protocolos apropriados e o suporte de *middleware* associado;
- o *middleware* e a plataforma subjacentes sejam configurados de forma a fornecer o nível correto de suporte para a configuração de componente (por exemplo, no CORBA, isso incluiria configurar o POA);
- os serviços do sistema distribuído associado sejam configurados de forma a fornecer o nível de segurança correto, suporte para transações, etc.

Também são fornecidas ferramentas para interpretar os descritores de distribuição e para garantir a correta implantação de determinada arquitetura física.

8.5 Estudos de caso: Enterprise JavaBeans e Fractal

A vantagem dos servidores de aplicativo é que eles fornecem amplo suporte para um estilo de programação distribuída – a estratégia de três camadas físicas explicada anteriormente – e a maior parte das complexidades associadas à programação distribuída fica oculta para o usuário. As desvantagens são que a estratégia é prescritiva e pesada – prescritiva no sentido de que a estratégia impõe esse estilo de arquitetura de sistemas em particular, e pesada porque os servidores de aplicativo são sistemas de *software* grandes e complexos, que inevitavelmente apresentam uma sobrecarga em termos de requisitos de desempenho e recursos. A estratégia funciona melhor em máquinas servidoras de alta capacidade.

Para contra-atacar isso, um estilo de programação com componentes mais despojado e mínimo também é adotado nos sistemas distribuídos. Referimos-nos a esse estilo como *modelos de componente leves*, para distingui-los das arquiteturas de servidor de aplicação, muito mais pesadas. Nesta seção, apresentaremos dois estudos de caso de tecnologias de componente: Enterprise JavaBeans, um exemplo importante da estratégia de servidor de aplicação, e Fractal, um exemplo de arquitetura de componente leve.

8.5.1 Enterprise JavaBeans

Enterprise JavaBeans (EJB) [java.sun.com XII] é uma especificação de arquitetura de componente gerenciada no lado do servidor e um elemento importante da plataforma Java, Enterprise Edition (Java EE), um conjunto de especificações para programação cliente-servidor. Outras especificações incluem RMI Java e JMS, apresentadas em outras partes, do livro (nos Capítulos 5 e 6, respectivamente).

O EJB é definido como *modelo de componente no lado do servidor*, pois suporta o desenvolvimento do estilo clássico de aplicações, em que números potencialmente grandes de clientes interagem com diversos serviços construídos por meio de componentes ou de configuração de componentes. Os componentes, conhecidos como *beans* no EJB, destinam-se a capturar a lógica da aplicação (ou do negócio), conforme definida no Capítulo 2, com o EJB também suportando a separação entre essa lógica da aplicação e seu armazenamento persistente em um banco de dados de *back-end*. Em outras palavras, o EJB fornece suporte direto para a arquitetura de três camadas físicas apresentada na Seção 2.3.2.

O EJB é *gerenciado*, no sentido de que o padrão contêiner, apresentado anteriormente (Seção 8.4), é usado para fornecer suporte para importantes serviços de sistemas distribuídos, incluindo transações, segurança e ciclo de vida. Normalmente, o contêiner injeta chamadas apropriadas para os serviços associados, a fim de fornecer as propriedades exigidas, sendo que o uso de um gerenciador de transação ou de serviços de segurança fica completamente oculto do desenvolvedor dos *beans* associados (*gerenciados pelo contêiner*). Também é possível ao desenvolvedor de *beans* ter mais controle sobre essas operações (*gerenciadas por beans*).

O objetivo do EJB é manter uma forte separação entre as várias funções envolvidas no desenvolvimento de aplicações distribuídas. A especificação EJB identifica as seguintes funções principais:

- o *provedor de bean*, que desenvolve a lógica da aplicação do componente (ou componentes);
- o *montador de aplicação*, que monta os *beans* nas configurações de aplicação;
- o *distribuidor*, que pega determinada montagem de aplicação e garante que ela seja corretamente distribuída em determinado ambiente operacional;
- o *provedor de serviço*, que é um especialista em serviços de sistema distribuído fundamentais, como gerenciamento de transação, e estabelece o nível de suporte desejado nessas áreas;
- o *provedor de persistência*, que é um especialista em mapear dados persistentes nos bancos de dados subjacentes e em gerenciar essa relação em tempo de execução;
- o *provedor de contêiner*, que complementa as duas funções anteriores e é responsável por configurar os contêineres corretamente, com o nível de suporte para sistemas distribuídos exigido em termos das propriedades não funcionais relacionadas, por exemplo, às transações e à segurança, assim como o suporte para persistência desejado;

- o *administrador de sistema*, que é responsável por monitorar a distribuição em tempo de execução e por fazer ajustes para garantir sua operação correta.

Note que o EJB é uma arquitetura de componente *pesada*, no sentido apresentado anteriormente. Há complexidade de *software* significativa, associada particularmente ao gerenciamento de contêineres. Assim, a estratégia é prescritiva e destinada apenas a certos tipos de aplicação. Conforme mencionado anteriormente, o EJB é particularmente adequado a aplicações que seguem a arquitetura de três camadas físicas, baseada em um banco de dados de *back-end* acessado por meio de uma interface de serviço oferecida pela camada física do meio (a lógica da aplicação). Por exemplo, esse estilo de arquitetura é comum em muitas aplicações de comércio eletrônico (*e-Commerce*), em que o banco de dados mantém informações sobre itens em estoque, preços e disponibilidade, enquanto a camada física do meio oferece interfaces para examinar o estoque e comprar itens selecionados. Normalmente, esses sistemas são grandes e complexos, exigindo suporte em termos de serviços de sistema distribuído; assim, a sobrecarga associada ao gerenciamento do contêiner é plenamente justificada. Como motivação, e para ilustrar o uso do EJB neste cenário, usaremos o exemplo de uma loja eletrônica (*e-Shop*) ao longo de toda esta seção. Outros tipos de aplicação não seguirão esse padrão e, assim, o EJB é uma tecnologia inadequada para tais aplicações. Exemplos incluem estruturas *peer-to-peer*, que simplesmente não seguem esse modelo de camadas físicas, e aplicações mais leves, executadas em dispositivos embarcados, nos quais a sobrecarga do EJB não se justifica.

Nesta seção, abordaremos os recursos do EJB 3.0 [java.sun.com XII], lançado em 2006. Um número grande de implementações de EJB 3.0 está disponível, tanto comercialmente como em consórcios de código-fonte aberto. Os principais exemplos incluem Spring, JBoss, JOnAS e Glassfish.

O modelo de componente do EJB • No EJB, um *bean* é um componente que oferece uma ou mais *interfaces do negócio* para clientes em potencial desse componente, em que as interfaces podem ser *remotas*, exigindo o uso de um *middleware* de comunicação apropriado (como RMI ou JMS), ou *local*, no caso em que são possíveis vínculos mais diretos e, assim, mais eficientes. Voltando à terminologia apresentada na Seção 8.4, uma interface do negócio é equivalente a uma *interface fornecida* (vamos ver como o EJB suporta interfaces exigidas a seguir, na subseção sobre injeção de dependência). Determinado *bean* é representado pelo conjunto de interfaces do negócio remotas e locais, junto a uma *classe de bean* associada que implementa as interfaces. Dois estilos principais de *bean* são suportados na especificação EJB 3.0:

Beans de sessão: um *bean* de sessão é um componente que implementa uma tarefa específica dentro da lógica da aplicação de um serviço, por exemplo, fazer uma compra em nossa aplicação *eShop*. Um *bean* de sessão persiste pela duração de um serviço e mantém uma conversa contínua com o cliente enquanto durar a sessão. Os *beans* de sessão podem ser *com estado*, mantendo o estado da conversa associado (como o *status* atual da transação de comércio eletrônico) ou *sem estado*, no caso em que nenhum estado é mantido. Os *beans* de sessão com estado implicam uma conversa com um único cliente e mantêm o estado dessa conversa. Em contraste, os *beans* sem estado podem ter muitas conversas concomitantes com diferentes clientes. O estado associado aos *beans* com estado pode ser persistente ou não, conforme discutiremos a seguir.

Beans baseados em mensagens: os clientes interagem com *beans* de sessão usando invocação local ou remota. Ao longo de todo o livro, temos visto que outros para-

digmas de comunicação também são importantes para o desenvolvimento de sistemas distribuídos, incluindo os paradigmas de comunicação indireta. O conceito de *bean* baseado em mensagens foi introduzido no EJB 2.0 para suportar comunicação indireta e, em particular, a possibilidade de interagir com os componentes usando filas de mensagens ou tópicos, complementando diretamente a funcionalidade oferecida pelo JMS (lembre-se de que filas e tópicos são entidades de primeira classe no JMS, representando intermediários alternativos para mensagens – consulte a Seção 6.4.3). Em um *bean* baseado em mensagens, uma interface do negócio será concretizada como uma interface estilo receptora, refletindo a natureza baseada em eventos do *bean* associado.

POJOs e anotações • A tarefa de programação no EJB foi significativamente simplificada pelo uso de *Enterprise JavaBeanPOJOs* (*Plain Old Java Objects* – *objetos Java puros*), junto a *anotações Enterprise JavaBean* da linguagem Java. Um *bean* (que é a implementação das interfaces do negócio do *bean*) é um objeto Java puro: ele consiste na lógica da aplicação escrita simplesmente em Java, sem que nenhum outro código relacionado seja um *bean*. Então, anotações são usadas para garantir o comportamento correto no contexto do EJB. Em outras palavras, um *bean* é um POJO complementado com anotações.

As anotações foram introduzidas no Java 1.5 como um mecanismo para associar metadados a pacotes, classes, métodos, parâmetros e variáveis. Esses metadados podem, então, ser usados por estruturas para garantir que o comportamento ou a interpretação correta seja associada a essa parte do programa. Como exemplo, anotações são usadas para introduzir um *bean* de um estilo específico. A seguir, estão exemplos de definições de *bean* anotados (representando os principais estilos de *bean* no EJB 3.0):

```
@Stateful public class eShop implements Orders {...}
@Stateless public class CalculatorBean implements Calculator {...}
@MessageDriven public class SharePrice implements MessageListener {...}
```

As anotações também são usadas para indicar se as interfaces do negócio são remotas (*@Remote*) ou locais (*@Local*). O exemplo a seguir introduz a interface *Orders* como uma interface remota e a interface *Calculator* de *CalculatorBean* apenas como uma interface local:

```
@Remote public interface Orders {...}
@Local public interface Calculator {...}
```

Conforme ficará claro, as anotações são usadas por todo o EJB, fornecendo uma especificação sobre como um programa deve ser interpretado em um contexto EJB.

Na descrição a seguir, vamos desenvolver o exemplo de loja eletrônica como uma ilustração do uso extensivo de anotações na programação de objetos *bean* (neste caso, um *bean* de sessão).

Contêineres Enterprise JavaBean no EJB • O EJB adota uma estratégia baseada em contêineres, conforme descrito na Seção 8.4. Os *beans* são distribuídos em contêineres, e estes fornecem gerenciamento de sistema distribuído implícito usando interceptação. Assim, o contêiner fornece as políticas necessárias em áreas que incluem gerenciamento de transação, segurança, persistência e gerenciamento de ciclo de vida, permitindo que o desenvolvedor de *bean* se concentre exclusivamente na lógica da aplicação. Portanto, os contêineres devem ser configurados com o nível de suporte necessário. Na versão atual, o EJB é previamente configurado com as políticas padrão comuns, e o desenvolvedor só

precisa tomar medidas se esses padrões forem insuficientes (o que é referido como *configuração por exceção* na especificação [java.sun.com XIII]).

Um número significativo de anotações são definidas para controlar os vários aspectos mencionados anteriormente. Ilustraremos seu uso enfocando o gerenciamento de transações do Enterprise JavaBean e incentivamos o leitor a examinar também as especificações do EJB 3.0 para mais exemplos. As transações serão apresentadas nos Capítulos 16 e 17. Em resumo, contudo, sua função é garantir que todos os objetos (ou, neste contexto, componentes) gerenciados por um único servidor (ou por vários servidores, no caso de transações distribuídas) permaneçam em um estado consistente, apesar do acesso concorrente de vários clientes e no caso de falha do servidor. Elas conseguem fazer isso por permitir que uma sequência de operações seja executada *de forma atômica*, no sentido de que a sequência de operações termine com êxito, de uma maneira que esteja livre da interferência de outro acesso concorrente, ou que a sequência não tenha nenhum efeito na presença de uma falha (como no caso de um colapso do servidor). Voltando ao nosso exemplo de loja eletrônica, um mecanismo de transação garantirá, por exemplo, que duas compras concomitantes não resultem na venda do mesmo item e que um colapso do servidor não permita que o sistema entre em um estado inconsistente, em que um item foi pago, mas não consignado ao comprador.

Os mecanismos para executar transações são relativamente complexos; portanto, dois capítulos são dedicados a esse assunto, posteriormente no livro. Contudo, a ideia geral é relativamente simples, e um entendimento intuitivo será suficiente para se entender como o EJB gerencia transações. O principal ponto a ser lembrado é que as transações se referem a sequências de operações e que as sequências devem ser claramente identificadas para que o serviço de gerenciamento de transação faça seu trabalho. No EJB, as transações se aplicam igualmente a qualquer estilo de *bean*, incluindo *beans* de sessão e *beans* baseados em mensagens.

A primeira coisa a ser declarada é se as transações associadas a um *bean* do negócio devem ser gerenciadas por *bean* ou por contêiner. Isso é conseguido pela associação das anotações a seguir com a classe associada, respectivamente:

`@TransactionManagement (BEAN)`

`@TransactionManagement (CONTAINER)`

As transações gerenciadas por *beans* são as mais simples de entender. Nesse caso, o desenvolvedor de *bean* é responsável por identificar explicitamente a sequência de operações a ser incluída dentro da transação. Isso é conseguido por se incluir explicitamente dois métodos da interface Java `javax.transaction.UserTransaction` – os métodos `UserTransaction.begin` e `UserTransaction.commit` – dentro do código do *bean*. Isso pode ser usado no lado cliente ou servidor de uma interação. O trecho de código a seguir ilustra o uso dessa estratégia gerenciada por *bean* no exemplo de loja eletrônica:

`@Stateful`

`@TransactionManagement (BEAN)`

`public class eShop implements Orders {`

`@Resource javax.transaction.UserTransaction ut;`

`public void MakeOrder (...) {`

`ut.begin ();`

`...`

`ut.commit ();`

`}`

`}`

Atributo	Política
<i>REQUIRED</i>	Se o cliente tem uma transação associada em execução, executa dentro dessa transação; caso contrário, inicia uma nova transação.
<i>REQUIRES_NEW</i>	Sempre inicia uma nova transação para essa invocação.
<i>SUPPORTS</i>	Se o cliente tem uma transação associada, executa o método dentro do contexto dessa transação; se não tiver, a chamada prossegue sem qualquer suporte para transação.
<i>NOT_SUPPORTED</i>	Se o cliente chama o método dentro de uma transação, essa transação é suspensa antes da chamada do método e, posteriormente, retomada – isto é, o método invocado é excluído da transação.
<i>MANDATORY</i>	O método associado deve ser chamado dentro de uma transação de cliente; se não for, uma exceção é disparada.
<i>NEVER</i>	Os métodos associados não devem ser chamados dentro de uma transação de cliente; se isso for tentado, uma exceção será disparada.

Figura 8.14 Atributos de transação no EJB.

Contudo, até certo ponto, isso vai contra o espírito da estratégia de contêiner, pois exige a inclusão de código relacionado à transação dentro do *bean*. A alternativa, transação gerenciada por contêiner, evita a necessidade desse código explícito, permitindo que o contêiner determine quando vai começar e terminar uma transação. Isso é obtido por meio da associação de determinada política de *demarcação* à execução do *bean*. Novamente, isso é conseguido de forma declarada por se associar uma anotação apropriada a determinado método dentro da classe de *bean*. Por exemplo, considere o trecho de código a seguir:

```
@Stateful public class eShop implements Orders {  
    ...  
    @TransactionAttribute(REQUIRED)  
    public void MakeOrder (...) {  
        ...  
    }  
}
```

Isso mostra a associação da política *REQUIRED* ao método *MakeOrder*. Essa política diz que o método associado deve ser executado dentro de uma transação. Para entender essa política, é necessário se dar conta de que uma transação pode ser iniciada pelo chamador ou ser a responsabilidade do próprio *bean*. A política *REQUIRED* inicia uma nova transação, se necessário – isto é, se o chamador não fornecer um contexto de transação indicando que o trabalho já está sendo feito dentro de uma transação. Essa e outras políticas estão resumidas na Figura 8.14.

Note que, por padrão, as transações são gerenciadas por contêiner no EJB.

Injeção de dependência: o exemplo anterior também ilustra mais uma função importante dos contêineres, a *injeção de dependência do Enterprise JavaBean*. A injeção de dependência é um padrão comum na programação, com a qual um agente externo, neste caso um contêiner, é responsável por gerenciar e solucionar as relações entre um componente e suas dependências (as interfaces exigidas, na terminologia da Seção 8.4). Em particular, no EJB 3.0, um componente se refere a uma dependência usando uma anotação, e o

contêiner é responsável por solucionar essa anotação e garantir que, em tempo de execução, o atributo associado se refira ao objeto correto. Normalmente, isso é implementado pelo contêiner usando reflexão.

Por exemplo, no trecho de código anterior, a anotação *@Resource* indica uma dependência desse componente em relação a um objeto que implementa a interface *UserTransaction*. Isso simplesmente deve existir para que a configuração faça sentido. A injeção de dependência tanto sinaliza essa dependência como garante que, quando a configuração de componente correta é implementada, o atributo *ut* se refira ao recurso externo correto.

Interceptação em Enterprise JavaBean • A especificação Enterprise JavaBeans permite ao programador interceptar dois tipos de operação em *beans* para alterar seus comportamentos padrão:

- chamadas de método associadas a uma interface do negócio;
- eventos de ciclo de vida.

Veremos cada um por sua vez, a seguir.

Interceptação de métodos: esse mecanismo é usado quando é necessário associar uma ação em particular ou um conjunto de ações a uma chamada recebida em uma interface do negócio. Isso se aplica igualmente às invocações recebidas em um *bean* de sessão ou a eventos recebidos em um *bean* baseado em mensagens. Conforme já vimos, a interceptação é usada amplamente na arquitetura EJB para fornecer gerenciamento implícito. Isso permite ao desenvolvedor da aplicação estender o uso de interceptação a preocupações mais específicas do domínio, não fornecidas pelo contêiner.

Considere o exemplo da loja eletrônica (*eShop*). Suponha que, dentro da loja eletrônica, haja a necessidade de implementar o registro de todas as operações realizadas no sistema, por exemplo, para propósitos de auditoria. A interceptação permite ao programador introduzir esse serviço sem alterar a lógica da aplicação contida no *bean*. Como um segundo exemplo, o mecanismo de interceptação poderia ser usado para impedir que certos clientes fizessem compras na loja eletrônica (por exemplo, caso estejam em débito com pagamentos anteriores).

Existem várias maneiras de associar interceptadores a determinado *bean*, incluindo associar uma classe de interceptação a determinada classe de *bean* ou método individual (usando a anotação *@Interceptors*), ou associar um método de interceptação a determinada classe (usando a anotação *@AroundInvoke*). Por simplicidade, abordaremos este último mecanismo e voltaremos ao nosso exemplo de loja eletrônica.

```
@Stateful
public class eShop implements Orders {
    public void MakeOrder (...) {
        ...
    }
    @AroundInvoke
    public Object log(InvocationContext ctx) throws Exception {
        System.out.println ("The following method was invoked: " +
                           ctx.getMethod().getName());
        return invocationContext.proceed();
    }
}
```


Assinatura	Uso
<code>public Object getTarget();</code>	Retorna a instância de <i>bean</i> associada à invocação ou ao evento recebido.
<code>public Method getMethod();</code>	Retorna o método que está sendo invocado.
<code>public Object[] getParameters();</code>	Retorna o conjunto de parâmetros associado ao método do negócio interceptado.
<code>public void setParameters(Object[] params);</code>	Permite que o conjunto de parâmetros seja alterado pelo interceptador, supondo que a exatidão de tipo seja mantida.
<code>public Object proceed() throws Exception;</code>	A execução passa para o próximo interceptador do encadeamento (se houver) ou para o método que foi interceptado.

Figura 8.15 Contextos de invocação no EJB.

A anotação `@AroundInvoke` introduz um interceptador na classe de *bean* *eShop*. O método do interceptador deve ter a seguinte sintaxe:

Objeto <nomeMétodo>(javax.ejb.ContextoInvocação)

Então, esse método é chamado quando qualquer um dos métodos do negócio é chamado em *eShop*. O parâmetro associado aumenta significativamente os recursos dos interceptadores, fornecendo metadados associados à invocação que está sendo interceptada (por exemplo, referências ao *bean*, ao método invocado e aos parâmetros reais associados à invocação) e também recursos limitados a interceder – isto é, para alterar os parâmetros antes que o método seja executado. A última linha do método, a chamada para *proceed*, retorna o controle para o método interceptado (ou para o próximo interceptador do encadeamento, caso mais de um interceptador esteja definido).

Os principais métodos associados ao contexto de invocação estão resumidos na Figura 8.15.

Interceptação de eventos de ciclo de vida: um mecanismo semelhante pode ser usado para interceptar e reagir a eventos de ciclo de vida associados a um componente. Em particular, a especificação EJB permite a um desenvolvedor de *bean* associar interceptadores à criação e à exclusão de componentes, usando as seguintes anotações, respectivamente:

`@PostConstruct`

`@PreDestroy`

As anotações são associadas a determinados métodos na classe de *bean*, com o efeito de que esses métodos serão chamados quando o evento de ciclo de vida associado acontecer. Por exemplo, no trecho de código a seguir da classe *eShop*, *TidyUp* será chamado imediatamente antes que o componente seja destruído:

```
@Stateful
public class eShop implements Orders {
    ...
    public void MakeOrder (...) {...}
    ...
    @PreDestroy void TidyUp() {... }}
}
```

Essa anotação é geralmente usada para liberar quaisquer recursos correntemente em uso pela classe *eShop*, por exemplo arquivos abertos ou soquetes associados à implementação de *eShop*. Analogamente, *TidyUp* poderia ser usado para garantir que dados importantes associados a *eShop* sejam escritos no banco de dados de *back-end*.

Se o *bean* tem estado, também é possível capturar eventos de ativação e passivação, usando *@PostActivate* e *@PrePassivate*. Novamente, isso permite que ações importantes sejam executadas em associação com esses eventos de ciclo de vida – por exemplo, garantindo que o estado da conversa associado a uma sessão seja armazenado no banco de dados antes que o *bean* seja passivado.

As anotações são usadas ao longo de todo o EJB 3.0 para fornecer um modelo de programação consistente e simples, por meio do qual os desenvolvedores de componente constróem a lógica da aplicação em POJOs e, então, decoram isso, onde for apropriado, com anotações de meta-nível adicionais que são interpretadas pela estrutura de contêiner.

8.5.2 Fractal

Conforme mencionado anteriormente, Fractal é um modelo de componente leve que traz as vantagens da programação baseada em componentes para o desenvolvimento de sistemas distribuídos [Bruneton *et al.* 2006, fractal.ow2.org I]. O Fractal fornece suporte para *programação com interfaces*, com os benefícios associados em termos da separação entre interface e implementação (benefícios também apresentados pelos objetos distribuídos). Contudo, o Fractal vai além e suporta a *representação explícita* da arquitetura de *software* do sistema, evitando o problema das dependências implícitas discutido na Seção 8.4. A estratégia é deliberadamente mínima, sem nenhum suporte para funcionalidade adicional relacionada a componentes, como a distribuição, o padrão contêiner completo ou o modelo de programação enriquecido oferecido pelos servidores de aplicação. O Fractal é usado para a construção de sistemas de *software* mais complexos (incluindo sistemas de *middleware*, conforme discutido a seguir), usando o modelo de componente como bloco de construção básico, resultando em *software* que tem uma arquitetura baseada em componentes clara e que é *configurável* e também *reconfigurável* em tempo de execução, para satisfazer o ambiente operacional e os requisitos atuais.

O Fractal define um modelo de programação e, como tal, é agnóstico quanto a linguagem de programação. Várias implementações desse modelo estão disponíveis em diversas linguagens diferentes, incluindo:

- Julia e AOKell (baseadas em Java, com a última oferecendo também suporte para programação orientada a aspectos);
- Cecilia e Think (baseadas em C);
- FracNet (baseada em .Net);
- FracTalk (baseada em Smalltalk);
- Julio (baseada em Python).

Julia e Cecilia são tratadas como as implementações de referência do Fractal.

O Fractal é suportado pelo consórcio OW2 [www.ow2.org], uma comunidade de *software* de código-fonte aberto para *middleware* de sistemas distribuídos que estimula e promove a filosofia baseada em componentes para a construção de tal *software*. Até hoje, o Fractal tem sido usado na construção de uma ampla variedade de plataformas de *middleware*, incluindo Think (um núcleo de sistema operacional configurável), DREAM

(uma plataforma de *middleware* que suporta várias formas de comunicação indireta), Jasmine (uma ferramenta que suporta monitoramento e gerenciamento de plataformas SOA), GOTM (que oferece gerenciamento de transação flexível) e Proactive (uma plataforma de *middleware* para computação de Grade – *Grid*). O Fractal também é a base do GCM (Grid Component Model), que tem sido influente no desenvolvimento de padrões ETSI associados [Baude *et al.* 2009]. Mais detalhes sobre todos esses projetos podem ser encontrados no *site* do OW2 [www.ow2.org].

Note que alguns outros modelos de componente leves foram desenvolvidos especificamente para sistemas distribuídos. Apresentaremos dois – OpenCOM e OSGI – no quadro a seguir.

O modelo de componente básico • No Fractal, um componente oferece uma ou mais interfaces, com dois tipos disponíveis:

- *interfaces de servidor*, que suportam as invocações recebidas (equivalentes às interfaces fornecidas na terminologia da Seção 8.4);
- *interfaces de cliente*, que suportam as invocações enviadas (equivalentes às interfaces exigidas).

Uma interface é uma implementação de um *tipo de interface*, o qual define as operações suportadas por essa interface.

Vínculos no Fractal: para permitir a composição, o Fractal suporta *vínculos* entre interfaces. Dois estilos de vínculo são suportados pelo modelo:

Vínculos primitivos: o estilo de vínculo mais simples é o *primitivo*, que é um mapeamento direto entre uma interface de cliente e uma interface de servidor dentro do mesmo espaço de endereçamento, supondo que os tipos sejam compatíveis. Os vínculos primitivos podem ser implementados eficientemente em determinado ambiente de linguagem, por exemplo, por meio de referências de objeto diretas.

Vínculos compostos: o Fractal também suporta *vínculos compostos*, que são arquiteturas de *software* arbitrariamente complexas (isto é, consistindo em componentes e vínculos) que implementam a comunicação entre várias interfaces, potencialmente em máquinas diferentes. Por exemplo, se você estivesse implementando uma conexão CORBA no Fractal, o vínculo seria formado por componentes representando os elementos arquitetônicos básicos no CORBA, incluindo *proxies*, o núcleo ORB, adaptadores de objeto, esqueletos e serventes (espelhando a arquitetura da Figura 8.5).

Os próprios vínculos compostos são componentes no Fractal, e isso é importante por dois motivos:

- Um sistema desenvolvido usando Fractal é totalmente configurável em termos dos componentes e de suas interconexões. Por exemplo, pode ser estabelecida uma configuração na qual os componentes interagem usando um vínculo composto, implementando qualquer um dos paradigmas de comunicação discutidos nos Capítulos 5 e 6 (invocação remota ou indireta, ponto a ponto ou de vários participantes, etc.). Se determinado paradigma de comunicação ainda não é fornecido, ele pode ser desenvolvido no Fractal e, então, disponibilizado como um componente para futuros desenvolvedores.

OpenCOM • OpenCOM [Coulson *et al.* 2008] é um modelo de componente leve com objetivos muito parecidos com os do Fractal. O OpenCOM é um modelo de componente mínimo e aberto, projetado para ser independente de domínio e de ambiente operacional; ou seja, a tecnologia de componentes é suficientemente flexível para ser aplicada em qualquer contexto, incluindo áreas exigentes, como redes de sensor sem fio com recursos limitados. O OpenCOM também é projetado para oferecer sobrecarga insignificante em termos de requisitos de desempenho e de memória, permitindo seu uso em situações em que o desempenho é fundamental – por exemplo, em implementações de roteador.

A arquitetura global do OpenCOM consiste em um núcleo mínimo, suportando operações de componente básicas que incluem carregamento e descarregamento de um componente e o vínculo de componentes. Isso, então, é melhorado com extensões refletivas e de plataforma opcionais, as quais suportam o carregamento dinâmico de recursos refletivos. Além disso, diferentes modelos servem de base para as operações importantes da plataforma, incluindo a semântica de carregamento e vínculo. Portanto, as extensões são conceitualmente semelhantes aos controladores no Fractal.

O OpenCOM tem sido usado no desenvolvimento de uma variedade de plataformas de *middleware* experimentais, incluindo ReMMoC [Grace *et al.* 2003], que oferece descoberta de serviço em ambientes de computação ubíquos altamente heterogêneos (veja o Capítulo 19), e GridKIT [Grace *et al.* 2008], uma estrutura de *middleware* experimental, altamente configurável e reconfigurável para computação em Grade, que também apresenta uma estrutura para a construção de redes virtualizadas arbitrárias, incluindo redes de sobreposição *peer-to-peer* estruturadas e não estruturadas.

OSGi • OSGi [www.osgi.org] é uma especificação de plataforma de *middleware* baseada em Java, gerenciada pela organização de padrões abertos, a OSGi Alliance. Existem várias implementações dessa especificação, incluindo Equinox, Knopflerfish, Felix e Concierge. A plataforma suporta a distribuição e o subsequente gerenciamento de ciclo de vida e adaptação de sistemas de *software* modulares, organizados como *pacotes* de comunicação (semelhantes aos componentes). Os pacotes se comunicam por meio de uma ou mais interfaces de serviço, com os serviços publicados em um registro, suportando, assim, o vínculo dinâmico. Como a unidade do gerenciamento de ciclo de vida, determinado pacote pode ser instalado, iniciado, ativado, parado e desinstalado. Os pacotes também podem ser distribuídos dinamicamente em tempo de execução e os já existentes podem ser atualizados. O OSGi foi desenvolvido originalmente para a programação de *gateways* de serviço (daí o nome original, *Open Service Gateway* i), mas agora é usado em uma grande variedade de domínios de aplicação, incluindo a programação de telefones móveis, como *middleware* para computação em Grade e também como arquitetura de *plug-in* no IDE (Integrated Development Environment – ambiente de desenvolvimento integrado) Eclipse, uma estrutura multilinguagem popular para desenvolvimento de *software*.

O OSGi tem como objetivo a distribuição e o gerenciamento de configurações de *software* centralizadas, residindo, por exemplo, em um único dispositivo ou em um servidor. Também foi desenvolvida uma implementação distribuída de OSGi, R-OSGi [Rellermeyer *et al.* 2007]. Isso permite que arquiteturas de *software* sejam distribuídas em limites de serviço entre configurações de rede arbitrárias. O R-OSGi usa o padrão *proxy*, apresentado no Capítulo 2, para obter distribuição transparente nesses limites.

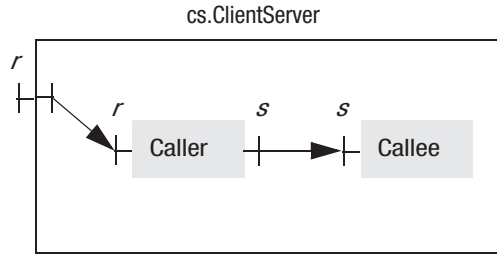


Figura 8.16 Um exemplo de configuração de componente no Fractal.

- Uma vez estabelecido, qualquer aspecto da arquitetura de *software* pode ser reconfigurado em tempo de execução, incluindo os vínculos compostos. A capacidade de adaptar estruturas de comunicação em tempo de execução é muito útil, por exemplo, para introduzir mais níveis de segurança ou alterar a implementação para que seja mais escalável à medida que o tamanho do sistema aumenta.

Veremos mais detalhes sobre o suporte para reconfiguração na seção sobre membranas e controladores, a seguir.

Composição hierárquica: o modelo de componente é hierárquico, pois um componente consiste em uma série de subcomponentes e vínculos associados, em que os próprios subcomponentes podem ser compostos. Por exemplo, o núcleo ORB no exemplo anterior poderia ser decomposto, dada sua inerente complexidade. A composição é suportada pela linguagem Fractal ADL (Architectural Description Language), a qual apresentamos por meio de um exemplo simples, que mostra a criação de um componente contendo dois subcomponentes que interagem como na estratégia cliente-servidor:

```
<definition name="cs.ClientServer">
  <interface name="r" role="server"
    signature="java.lang Runnable" />
  <component name="caller" definition="hw.CallerImpl" />
  <component name="callee" definition="hw.CalleeImpl" />
  <binding client="this.r" server="caller.r" />
  <binding client="caller.s" server="callee.s" />
</definition>
```

A Fractal ADL é baseada na XML. Esse exemplo mostra um componente *cs.ClientServer* com dois subcomponentes, *caller* e *callee*; vínculos são criados entre a interface de cliente, *this.r* (isto é, a interface *r* definida no componente contêiner *cs.ClientServer*) e a interface associada *caller.r* (a interface *r* definida no componente *caller*), e entre a interface de cliente *caller.s* e a interface de servidor correspondente *callee.s*. A configuração associada está ilustrada na Figura 8.16.

O Fractal também suporta *compartilhamento*, por meio do qual determinado componente pode ser compartilhado entre várias arquiteturas de *software*. Os desenvolvedores do Fractal argumentam que isso é necessário para representar fielmente as arquiteturas de sistema, incluindo o acesso aos recursos subjacentes que são fundamentalmente compartilhados, como uma conexão TCP, por exemplo. Como mais um exemplo, seria possível *callee* (o componente servidor) ser compartilhado entre várias configurações.

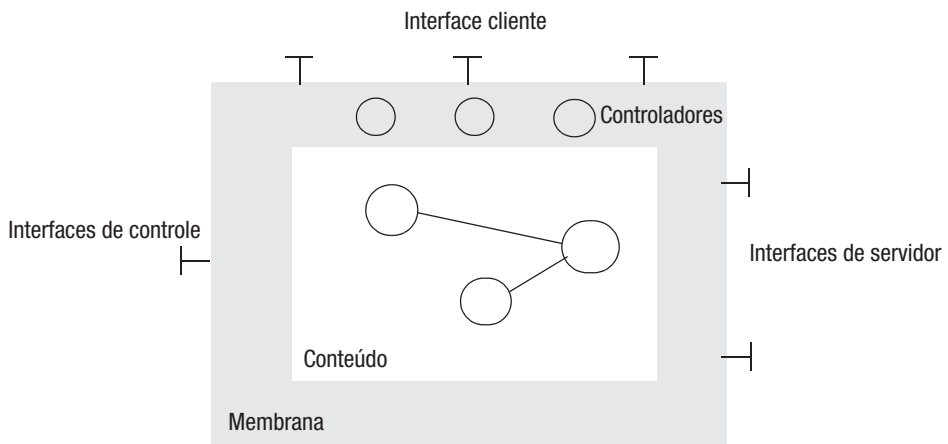


Figura 8.17 A estrutura de um componente do Fractal.

Membranas e controladores • Na implementação, um componente consiste em uma *membrana*, a qual define recursos de controle associados ao componente por meio de um conjunto de *controladores*, e também no *conteúdo* associado – os subcomponentes (e vínculos) que constituem sua arquitetura. As interfaces podem ser *internas* à membrana e, portanto, visíveis apenas para os componentes dentro da parte relativa ao conteúdo, ou *externas* e, portanto, visíveis para quaisquer outros componentes. Essa estrutura está ilustrada na Figura 8.17.

O conceito de membrana é fundamental para a estratégia do Fractal: uma membrana fornece um regime de controle configurável para o conjunto de componentes encapsulados (o conteúdo). Em outras palavras, o conjunto de controladores define os recursos de controle e a semântica associada desses componentes. Mudando o conjunto de controladores, mudamos também os recursos.

Os controladores podem ser usados para vários propósitos:

- Um dos principais usos dos controladores é na implementação de *gerenciamento de ciclo de vida*, incluindo as operações associadas à ativação e à passivação, como *suspend*, *resume* e *checkpoint*. Por exemplo, o Fractal suporta um controlador *LifeCycleController*, que possui três métodos, *startFc*, *stopFc* e *getFcState*, que implementam essas três funções, respectivamente. Isso é fundamental nos casos em que as reconfigurações da arquitetura de *software* subjacente estão sendo feitas em tempo de execução. Considere o exemplo simples da configuração cliente-servidor anterior e suponha que o servidor seja substituído dinamicamente por outro, aprimorado (talvez um que suporte *multithreading* para obter melhor desempenho de saída). Neste caso, para evitar inconsistências, é interessante suspender a configuração, substituir o componente *callee* por um novo e, então, retomar a configuração.
- Os controladores também oferecem recursos de reflexão (veja o Capítulo 2). Em particular, os recursos de *introspecção* são fornecidos por meio de duas interfaces, *Component* e *ContentController*, as quais suportam introspecção (descoberta dinâmica) das interfaces associadas a um componente e percorrem a arquitetura

```

public interface Component {
    Object[] getFcInterfaces ();
    Object getFcInterface (String itfName);
    Type getFcType ();
}

public interface ContentController {
    Object[] getFcInternalInterfaces ();
    Object getFcInterfaceInterface(String itfName);
    Component[] getFcSubComponents ();
    void addFcSubComponent (Component c);
    void removeFcSubComponent(Component c);
}

```

Figura 8.18 Interfaces *Component* e *ContentController* no Fractal.

tura de uma estrutura de componente composta, respectivamente. As interfaces completas dos dois controladores estão mostradas na Figura 8.18. Novamente, a introspecção é importante para suportar reconfiguração dinâmica. Voltando a nosso exemplo de cliente-servidor, é possível, por meio das interfaces anteriores, descobrir a arquitetura precisa da configuração de componente subjacente (neste caso, uma configuração simples, consistindo em dois componentes) e também garantir que um componente *callee* substituto suporte precisamente a mesma interface do antigo.

- Também podem ser introduzidos controladores para oferecer recursos de interceptação, espelhando a capacidade oferecida no EJB e relatada na Seção 8.5.1. A interceptação é um mecanismo poderoso, com muitos usos. Na seção sobre EJB, é fornecido um exemplo de uso de interceptação para implementar registro. Por exemplo, a interceptação poderia ser usada no exemplo de cliente-servidor para registrar todas as chamadas feitas pelo componente *caller*, de uma maneira completamente transparente tanto para *caller* como para *callee*. Outro uso de interceptação é na implementação de uma política de controle de acesso que só permita o prosseguimento de uma invocação se determinado principal tiver direitos de acesso a determinado recurso (veja a Seção 11.2.4).

Tendo estudado as funções relativas das membranas e dos controladores, agora é possível relacionar as membranas aos contêineres, apresentados na Seção 8.4, e no estudo de caso do EJB anterior. Assim como os contêineres, as membranas fornecem um lugar para a distribuição de componentes; as duas técnicas também suportam gerenciamento de sistemas distribuídos implícito, os contêineres fazendo chamadas implícitas para serviços dos sistemas distribuídos e as membranas por meio de seus controladores constituintes. Contudo, as membranas são significativamente mais flexíveis:

- Em termos de reflexão, o suporte pode variar de componentes tipo caixa preta, em que a estrutura interna fica escondida, estratégias nas quais são oferecidos recursos de introspecção limitados (descobrendo interfaces dinamicamente), até recursos de reflexão avançados que suportam introspecção total e fornecem suporte inerente para a subsequente adaptação das estruturas internas.
- Em termos de suporte para preocupações não funcionais, em um extremo, as membranas não podem fornecer mais do que um simples encapsulamento de

componentes (se, por exemplo, controladores mínimos forem implementados); no outro extremo, elas podem suportar gerenciamento de sistemas distribuídos completo de componentes, incluindo o suporte para transação e segurança, como em servidores de aplicativo, mas de maneira completamente configurável e re-configurável. Graças a essa flexibilidade inerente, o Fractal é referido como modelo de componente *aberto*.

Um exemplo funcional de implementação completa de Fractal pode ser encontrado no tutorial *online* [fractal.ow2.org II]. Esse exemplo mostra como o Fractal pode ser usado para implementar um servidor HTTP mínimo e configurável, chamado Comanche.

8.6 Resumo

Este capítulo examinou o projeto de soluções de *middleware* completas, baseadas em objetos distribuídos e componentes. Conforme ficará claro, elas representam uma evolução natural ao se pensar em tais abstrações de programação. Os objetos distribuídos são importantes para trazer as vantagens do encapsulamento e da abstração de dados para os sistemas distribuídos e também as ferramentas e técnicas associadas do campo do projeto orientado a objetos. Portanto, os objetos distribuídos representam um passo significativo em relação às estratégias anteriores, baseadas diretamente no modelo cliente-servidor. No entanto, na aplicação da estratégia de objeto distribuído, surgiram várias limitações significativas e elas foram apresentadas e analisadas neste capítulo. Em resumo, na prática, frequentemente é complexo demais usar soluções de *middleware* como o CORBA para aplicações e serviços distribuídos sofisticados, particularmente ao se lidar com propriedades avançadas de tais sistemas, como, por exemplo, a confiabilidade (tolerância a falhas e segurança).

As tecnologias de componente superam essas limitações por meio de sua separação de preocupações intrínseca entre lógica da aplicação e gerenciamento de sistemas distribuídos. A identificação explícita de dependências também ajuda a suportar a composição terceirizada de sistemas distribuídos. Este capítulo examinou a especificação EJB 3.0, que fez avanços ao simplificar o desenvolvimento de sistemas distribuídos por meio de uma estratégia que enfatiza o uso de objetos Java puros, com as complexidades gerenciadas declarativamente por meio do uso de anotações Java. Como vimos, tecnologias mais leves, como Fractal e OpenCOM, também foram introduzidas para trazer as vantagens da programação baseada em componentes para o desenvolvimento das próprias plataformas de *middleware*, com pouca sobrecarga adicional em termos de desempenho.

As tecnologias de componente são importantes para o desenvolvimento de aplicações distribuídas, mas assim como qualquer tecnologia, têm suas vantagens e desvantagens. Por exemplo, a estratégia de componentes é muito prescritiva e mais conveniente para aplicações naturalmente semelhantes às arquiteturas de três camadas físicas. Para oferecer uma perspectiva mais ampla sobre a variedade de plataformas de *middleware* disponíveis, os dois próximos capítulos examinarão estratégias alternativas, baseadas na adoção de padrões da Web (serviços Web) e sistemas *peer-to-peer*.

Exercícios

- 8.1 A *Task Bag* (sacola de tarefas) é um objeto que armazena pares (chave e valor). Uma chave é um *string* e um valor é uma sequência de bytes. Sua interface fornece os seguintes métodos remotos:

pairOut: com dois parâmetros, através dos quais o cliente especifica uma *chave* e um *valor* a ser armazenado.

pairIn: cujo primeiro parâmetro permite que o cliente especifique a *chave* de um par a ser removido da *Task Bag*. O valor no par é fornecido para o cliente por intermédio de um segundo parâmetro. Se nenhum par correspondente estiver disponível, uma exceção será lançada.

readPair: é igual a *pairIn*, exceto que o par permanece na *Task Bag*.

Usando a IDL do CORBA, defina a interface da *Task Bag*. Defina uma exceção que possa ser disparada quando qualquer uma das operações não puder ser executada. Sua exceção deve retornar um valor inteiro indicando o número do problema e um *string* descrevendo o problema. A interface da *Task Bag* deve definir um único atributo, fornecendo o número de tarefas na sacola.

página 341

- 8.2 Defina uma assinatura alternativa para os métodos *pairIn* e *readPair*, cujo valor de retorno indica quando nenhum par correspondente está disponível. O valor de retorno deve ser definido como um tipo enumerado cujos valores podem ser *ok* e *wait*. Discuta os méritos relativos das duas estratégias alternativas. Qual estratégia você usaria para indicar um erro como uma chave contendo caracteres inválidos?

página 342

- 8.3 Qual dos métodos na interface da *Task Bag* poderia ter sido definido como uma operação *oneway*? Forneça uma regra geral a respeito dos parâmetros e exceções de métodos *oneway*. De que maneira o significado da palavra-chave *oneway* difere do restante da IDL?

página 342

- 8.4 O tipo *union* da IDL pode ser usado para um parâmetro que precisará passar um entre poucos número de tipos. Utilize-o para definir o tipo de um parâmetro que às vezes é vazio e às vezes tem o tipo *Value*.

página 345

- 8.5 Na Figura 8.2, o tipo *All* foi definido como uma sequência de comprimento fixo. Redefina isso como um vetor de mesmo comprimento. Dê algumas recomendações quanto à escolha entre vetores e sequências em uma interface IDL.

página 345

- 8.6 A *Task Bag* se destina ao uso por clientes em cooperação, alguns dos quais adicionam pares (descrevendo tarefas) e outros os removem (e executam as tarefas descritas). Quando um cliente é informado de que não há um par correspondente disponível, ele não pode continuar seu trabalho até que um par se torne disponível. Defina uma interface de *callback* apropriada para uso nesta situação.

página 357

- 8.7 Descreva as modificações necessárias na interface da *Task Bag* para permitir o uso de *callbacks*.

página 357

- 8.8 Quais dos parâmetros dos métodos na interface da *Task Bag* são passados por valor e quais são passados por referência?

página 343

- 8.9 Use o compilador de IDL Java para processar a interface que você definiu no Exercício 8.1. Inspeção a definição das assinaturas dos métodos *pairIn* e *readPair* no equivalente em Java gerado da interface IDL. Procure também, na definição gerada, o argumento de valor dos métodos *pairIn* e *readPair*. Agora, dê um exemplo mostrando como o cliente invocará o método *pairIn*, explicando como adquirirá o valor retornado por meio do segundo argumento.

página 346

- 8.10 Dê um exemplo para mostrar como um cliente Java acessará o atributo que fornece o número de tarefas no objeto *Task Bag*. Sob que aspectos um atributo difere de uma variável de instância de um objeto?

página 345

- 8.11 Explique por que as interfaces de objetos remotos, em geral, e os objetos CORBA, em particular, não fornecem construtores. Explique como os objetos CORBA podem ser criados na ausência de construtores. *Capítulo 5 e página 355*
- 8.12 Redefina a interface da *Task Bag* do Exercício 8.1 na IDL, de modo que ela faça uso de uma *struct* para representar um *Par*, o qual consiste em uma *Chave* e em um *Valor*. Note que não há necessidade de usar um *typedef* para definir uma *struct*. *página 345*
- 8.13 Discuta as funções do repositório de implementações do ponto de vista da capacidade de mudança de escala e da tolerância a falhas. *página 350, página 351*
- 8.14 Até que ponto os objetos CORBA podem ser migrados de um servidor para outro? *página 350, página 351*
- 8.15 Explique exatamente como o *middleware* baseado em componentes em geral e o EJB, em particular, podem superar as principais limitações do *middleware* de objeto distribuído. Dê exemplos para ilustrar sua resposta. *páginas 358 a 364*
- 8.16 Discuta se a arquitetura do EJB seria conveniente para implementar um jogo *online* para muitos jogadores (um domínio de aplicação apresentado inicialmente no Capítulo 1, Seção 1.2.2). Quais seriam as vantagens e desvantagens de usar EJB nesse domínio? *página 364*
- 8.17 O Fractal seria uma escolha de implementação mais conveniente para o domínio do MMOG? Justifique sua resposta. *página 372*
- 8.18 Explique como a filosofia baseada em contêiner poderia ser adotada para fornecer transparência de migração para componentes distribuídos. *página 362*
- 8.19 Como você obteria o mesmo efeito no Fractal? *página 375*
- 8.20 Considere a implementação de RMI Java como um vínculo composto no Fractal. Discuta até que ponto tal vínculo pode ser configurável e reconfigurável. *Capítulo 5 e página 373*